THE OOP C++ BIBLE

Giáo Trình Khoa Học Máy Tính & Kiến Trúc Phần Mềm Hướng Đối Tượng

---

Phân Tích Chuyên Sâu C++11/14/17 Dưới Tầng Phần Cứng, Quản Trị Bộ Nhớ RAM, VTable & Cơ Chế Trình Biên Dịch

---

Chuyên Luận Đào Tạo Kỹ Sư Công Nghệ Thông Tin Cao Cấp

Tái hiện toàn diện cơ sở lý thuyết, mô hình kiến trúc bộ nhớ, các hệ hình lập trình, và giải thuật tối ưu hóa tài nguyên chuẩn mực quốc tế ISO/IEC 14882.

Tủ Sách Kỹ Thuật Máy Tính & Khoa Học Phần Mềm

Lưu Hành Học Thuật & Nghiên Cứu Chuyên Sâu — Năm 2026

Contents

1 Triết lý & Nền tảng (Philosophy & Foundations) 1
- 1.1 Sự Tiến Hóa Hệ Hình: Từ Ngôn Ngữ C Sang C++ . . . 1
- 1.2 Kiến Trúc Bộ Nhớ Tiến Trình (Process Memory Layout) . . . 2
- 1.3 Quản Lý Bộ Nhớ Động: So Sánh Đối Kháng malloc/free và new/delete 3
- 1.4 Con Trỏ, Tham Chiếu và Cuộc Cách Mạng Move Semantics (C++11) . 5
  - 1.4.1 Bản Chất Dưới Gầm Máy: Tham Chiếu Lvalue (Lvalue Reference) 5
  - 1.4.2 Hệ Thống Phân Loại Giá Trị (Value Categories) Trong C++11 5
  - 1.4.3 Tham Chiếu Rvalue (`T&&`) và Ngữ Nghĩa Di Chuyển (Move Semantics) . . . 6
  - 1.4.4 Quy Tắc Sụp Đổ Tham Chiếu (Reference Collapsing Rules) . . 7
- 1.5 Hệ Thống Ép Kiểu An Toàn Chuẩn C++11 . . . 8

2 Lớp & Đối tượng (Classes & Objects) 9
- 2.1 Tính Đóng Gói (Encapsulation) & Bộ Chỉ Định Quyền Truy Cập . . . 9
  - 2.1.1 Cơ Chế Phân Quyền Truy Cập . . . 9
  - 2.1.2 Khảo Sát Bộ Nhớ: Sự Căn Chỉnh Dữ Liệu (Memory Alignment & Padding) . . . 10
- 2.2 Bản Chất Con Trỏ this Dưới Tầng RAM & ABI Gọi Hàm . . . 11
- 2.3 Vòng Đời Khởi Tạo Đối Tượng (Constructors Architecture) . . . 12
  - 2.3.1 Member Initializer List vs Phép Gán Trong Thân Constructor . 12
  - 2.3.2 Ngăn Chặn Ép Kiểu Ngầm Định Với Từ Khóa `explicit` . . . 13
- 2.4 Hàm Hủy (Destructor) & Triết Lý Thiết Kế RAII . . . 14
- 2.5 Thành Viên Static, Const và Kỹ Thuật Hàm Bạn (Friend) . . . 14
  - 2.5.1 Thành Viên Tĩnh (Static Data & Methods) . . . 14
  - 2.5.2 Từ Khóa `mutable` — Ngoại Lệ Trong Phương Thức Hằng . . . 14
  - 2.5.3 Quan Hệ Bạn Bè (Friendship Architecture) . . . 15

3 Quản trị Bộ nhớ & Nạp chồng Toán tử (Memory & Overloading) 16
- 3.1 Sao Chép Nông (Shallow Copy) vs Sao Chép Sâu (Deep Copy) . . . 16
  - 3.1.1 Cơ Chế Phá Hủy Bộ Nhớ Của Shallow Copy (Double-Free Disaster) 16
  - 3.1.2 Giải Pháp Chuẩn Mực: Sao Chép Sâu (Deep Copy) . . . 17
- 3.2 Tiến Hóa Bộ Quy Tắc Quản Trị Tài Nguyên: Rule of 3, 5, 0 . . . 18
  - 3.2.1 Quy Tắc Bộ Ba Cổ Điển (The Rule of Three — C++98) . . . 18
  - 3.2.2 Quy Tắc Bộ Năm Hiện Đại (The Rule of Five — C++11) . . . 18
  - 3.2.3 Thành Ngữ Copy-and-Swap (The Copy-and-Swap Idiom) . . . 18

3.2.4 Quy Tắc Số Không (The Rule of Zero — Modern C++) 19
3.3 Nạp Chồng Toán Tử Toàn Diện (Operator Overloading Deep Dive) . . 20
3.3.1 Các Định Đề Bất Biến Về Toán Tử 20
3.3.2 Lựa Chọn Thiết Kế: Member Method vs Non-member Friend . 20
3.3.3 Toán Tử Chỉ Mục `operator[]` & Toán Tử Gọi Hàm `operator()` 21
3.3.4 Toán Tử Tăng Giảm: Tiền Tố (Prefix) vs Hậu Tố (Postfix) . . 21
3.4 Nạp Chồng Toán Tử new và delete & Kỹ Thuật Placement New 22
3.4.1 Nạp Chồng new/delete Cấp Lớp (Class-specific Allocators) . . . 22
3.4.2 Kỹ Thuật Khởi Tạo Tại Chỗ (Placement New) 22
3.5 Kiến Trúc Con Trỏ Thông Minh Chuẩn C++11 (Smart Pointers Architecture) 23
3.5.1 `std::unique_ptr<T>` — Quyền Sở Hữu Độc Quyền (Exclusive Ownership) 23
3.5.2 `std::shared_ptr<T>` — Quyền Sở Hữu Chia Sẻ (Shared Ownership) 23
3.5.3 `std::weak_ptr<T>` — Bẻ Gãy Vòng Lặp Tham Chiếu (Circular Reference) 24

4 Kế thừa (Inheritance) 25
4.1 Bản Chất Quan Hệ IS-A và HAS-A 25
4.1.1 Bố Cục Bộ Nhớ Đối Tượng Kế Thừa Đơn (Single Inheritance Memory Layout) 26
4.2 Ma Trận Biến Đổi Quyền Truy Cập Trong Kế Thừa 26
4.3 Vòng Đời Khởi Tạo và Hủy Kế Thừa 27
4.3.1 Trình Tự Thực Thi Khởi Tạo và Giải Phóng Bộ Nhớ 27
4.4 Hiện Tượng Che Khuất Tên (Name Hiding) vs Nạp Chồng 28
4.5 Upcasting, Downcasting và Hiểm Họa Cắt Xén Đối Tượng 29
4.5.1 Cắt Xén Đối Tượng (Object Slicing) 29
4.6 Đa Kế Thừa & Bài Toán Kim Cương (Diamond Problem) 30
4.6.1 Mô Hình Bài Toán Kim Cương 30

5 Đa hình & Liên kết Động (Polymorphism & Dynamic Binding) 32
5.1 Triết lý Đa hình & Cơ chế Liên kết (Binding) 32
5.2 Hàm Ảo, `override` và `final` trong C++11 33
5.2.1 Cơ chế Hàm Ảo (`virtual`) 33
5.2.2 Bộ Đôi Từ Khóa Ngữ Cảnh C++11: `override` và `final` 34
5.3 Phân tích Dưới Gầm Máy: Bảng Ảo (VTable) & Con trỏ vptr 35
5.3.1 Cơ Chế Điều Chỉnh Con Trỏ Thunk (Adjustor Thunk) 36
5.4 Hàm Thuần Ảo & Lớp Cơ Sở Trừu Tượng (Abstract Base Classes) . . 37
5.4.1 Bẫy Tinh Vi: Hàm Thuần Ảo Vẫn Có Thể Có Body! 37
5.4.2 Pure Virtual Destructor: Phải Luôn Có Body! 38
5.5 Destructor Ảo (Virtual Destructor) — Luật Sống Còn 38
5.6 Bẫy Tử Thần: Gọi Hàm Ảo Trong Constructor & Destructor 40

6 Khuôn mẫu & Lập trình Tổng quát (Templates & Generic Programming) 42
6.1 Khuôn Mẫu Hàm & Khuôn Mẫu Lớp (Function & Class Templates) . . 42
6.1.1 Cú pháp và Sự tương đương giữa `typename` và `class` 42

6.2 Quá Trình Biên Dịch Hai Pha (Two-Phase Lookup) & Dependent Types 43
6.2.1 Bắt Buộc Sử Dụng Từ Khóa `typename` Cho Kiểu Phụ Thuộc (Dependent Types) . . . 43
6.3 Cơ Chế Tạo Thực Thể & Hiện Tượng Phình To Mã (Code Bloat) . . . 44
6.3.1 Tại Sao Phải Viết Toàn Bộ Code Template Trong File Header (`.hpp`)? . . . 44
6.3.2 Hiện Tượng Phình To Mã Nguồn (Code Bloat) . . . 45
6.4 Tham Số Không Phải Kiểu (NTTP) & Template Template Parameters 46
6.4.1 Tham Số Không Phải Kiểu (Non-type Template Parameters - NTTP) . . . 46
6.5 Chuyên Biệt Hóa Khuôn Mẫu (Template Specialization) & Bẫy Herb Sutter . . . 46
6.5.1 Chuyên Biệt Hóa Toàn Phần & Một Phần (Full vs Partial Specialization) . . . 46
6.5.2 Bẫy Tử Thần Herb Sutter: Function Template Không Hỗ Trợ Partial Specialization! . . . 47
6.6 Các Tính Năng Khuôn Mẫu C++11 Hiện Đại . . . 47
6.6.1 Variadic Templates (Khuôn Mẫu Tham Số Biến Thiên) . . . 47
6.6.2 CRTP: Đa Hình Tĩnh (Static Polymorphism) . . . 48

7 Xử lý Ngoại lệ & Thư viện Chuẩn STL (Exceptions & Modern STL) 50
7.1 Cơ Chế Bắt Ngoại Lệ & Bẫy Object Slicing . . . 50
7.2 Giải Phóng Ngăn Xếp (Stack Unwinding) & Liên Minh RAII . . . 51
7.3 Từ Khóa `noexcept` & Tối Ưu Hóa STL Sống Còn Trong C++11 . . . 52
7.3.1 Cam Kết Bất Biến `noexcept` . . . 52
7.3.2 Bí Ẩn Tối Ưu Hiệu Năng Của `std::vector` Trong C++11 . . . 53
7.4 Ngoại Lệ Trong Constructor & Destructor — Những Vùng Đất Cấm . 53
7.4.1 Ngoại Lệ Trong Constructor: Hiểm Họa Rò Rỉ Con Trỏ Thô . . 54
7.4.2 Kỹ Thuật Function Try Block Trong Constructor . . . 54
7.4.3 Luật Cấm Tuyệt Đối: Ngoại Lệ Thoát Ra Khỏi Destructor . . . 55
7.5 Ba Cấp Độ Cam Kết An Toàn Ngoại Lệ (Exception Safety Guarantees) 55
7.5.1 Nghiên Cứu Điển Hình Kiến Trúc: Tại Sao `std::stack::pop()` Trả Về `void`? . . . 56
7.6 Kiến Trúc Tách Rời Trong Thư Viện Chuẩn C++ (STL) . . . 57
7.6.1 Kiến Trúc Tách Rời (Decoupled Architecture) . . . 57
7.6.2 Năm Hạng Bộ Lặp & Kỹ Thuật Tag Dispatching . . . 57
7.6.3 Kiến Trúc Quản Trị Bộ Nhớ: Bộ Cấp Phát Tùy Biến (Allocators & Rebind) . . . 58

Lời Kết: Con Đường Làm Chủ C++ Hiện Đại 59

CHƯƠNG 1

Triết lý & Nền tảng (Philosophy & Foundations)

Tiên Đề / Định Nghĩa Pháp Lập: Định Lý Nền Tảng: Nguyên Lý Không Chi Phí Phụ Trội (Zero-Overhead Principle)

Được phát biểu bởi Bjarne Stroustrup, nguyên lý cốt tử chi phối toàn bộ tiến trình tiến hóa của C++ khẳng định hai mệnh đề bất biến:

1. **Mệnh đề Tiêu trừ:** Những cơ chế ngôn ngữ hoặc tính năng trừu tượng mà một chương trình không sử dụng thì chương trình đó tuyệt đối không phải trả giá bằng bất kỳ chi phí thời gian thực thi (runtime CPU cycles) hay dung lượng bộ nhớ nào.
2. **Mệnh đề Tối ưu:** Khi một lập trình viên sử dụng một tính năng trừu tượng của ngôn ngữ, mã máy do trình biên dịch sinh ra có hiệu năng tối thiểu bằng, hoặc vượt trội hơn, mã nguồn viết thủ công bằng hợp ngữ hoặc ngôn ngữ bậc thấp.[a]

[a] Nguồn: OOP1-C to C++-New.pdf - Trang 3-8: Sự tiến hóa từ C lên C++ và triết lý thiết kế hệ thống

1.1 Sự Tiến Hóa Hệ Hình: Từ Ngôn Ngữ C Sang C++

Ngôn ngữ lập trình C (do Dennis Ritchie phát triển tại Bell Labs) đại diện cho đỉnh cao của hệ hình **Lập trình Thủ tục (Procedural Programming)**. Trong hệ hình này, kiến trúc phần mềm được phân tách thành hai thực thể độc lập:

- **Dữ liệu (Data):** Được biểu diễn dưới dạng các cấu trúc biến thô (`struct`) thụ động, không có cơ chế tự vệ.
- **Hàm xử lý (Functions):** Các khối lệnh thao tác trực tiếp lên dữ liệu thông qua việc truyền tham trị hoặc con trỏ.

Sự phân tách này bộc lộ giới hạn nghiêm trọng khi quy mô hệ thống phần mềm tăng lên hàng triệu dòng lệnh. Do thiếu tính đóng gói (*Encapsulation*), trạng thái nội tại của dữ liệu có thể bị xâm phạm hoặc làm sai lệch từ bất kỳ điểm nào trong không gian mã nguồn.

Sự ra đời của C++ (khởi đầu là *C with Classes*) đánh dấu cuộc cách mạng chuyển dịch sang hệ hình **Lập trình Hướng Đối tượng (Object-Oriented Programming - OOP)** và sau đó là **Lập trình Tổng quát (Generic Programming)**. C++ hợp nhất dữ liệu và các thao tác biến đổi trạng thái thành một thực thể duy nhất: **Lớp (Class)**.

Table 1.1: Ma trận so sánh các đặc tính kiến trúc giữa C và C++

Đặc Tính Kiến Trúc	**Ngôn Ngữ C (C99/C11)**	**Ngôn Ngữ C++ (C++11/14/17)**
Hệ hình chủ đạo	Lập trình Thủ tục / Hướng chức năng	Đa hệ hình: Hướng đối tượng, Tổng quát, Hàm
Cơ chế kiểm soát truy cập	Hoàn toàn không có (Toàn bộ là public)	Ba tầng: `public`, `protected`, `private`
Cấp phát bộ nhớ động	Hàm thư viện `malloc()`, `free()`	Toán tử ngôn ngữ `new`, `delete`, Smart Pointers
Quản trị vòng đời đối tượng	Thủ công, dễ rò rỉ bộ nhớ	Tự động hóa qua Constructor/Destructor (RAII)
Tính đa hình	Con trỏ hàm thủ công trong struct	Bảng hàm ảo tĩnh `vtable` và con trỏ `vptr`
Xử lý lỗi hệ thống	Mã lỗi trả về (*Error Codes*), biến `errno`	Cơ chế ném và bắt ngoại lệ (*Exceptions*), `noexcept`
Không gian tên	Toàn cục phẳng (*Flat namespace*), dễ xung đột	Phân cấp mô-đun qua `namespace`
Khả năng mở rộng toán tử	Không hỗ trợ	Nạp chồng toán tử toàn diện (*Operator Overloading*)

1.2 Kiến Trúc Bộ Nhớ Tiến Trình (Process Memory Layout)

Để nắm vững bản chất của hướng đối tượng, kỹ sư phần mềm phải hiểu rõ cách thức hệ điều hành (OS) và bộ vi xử lý (CPU) tổ chức không gian bộ nhớ ảo (Virtual Address Space). Trên các hệ điều hành hiện đại chạy trên kiến trúc x86-64, mỗi tiến trình người dùng (User-space Process) được cấp phát một không gian địa chỉ ảo 48-bit (tương đương 256 Terabytes), được phân tầng chặt chẽ từ địa chỉ thấp đến địa chỉ cao.

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Giải Phẫu Chi Tiết Các Phân Vùng Bộ Nhớ

1. **Text/Code Segment:** Chứa mã máy nhị phân của tất cả các hàm thành viên và thuật toán. Phân vùng này được nạp cờ bảo vệ **Read-Only và Executable** (`RX`) từ bảng trang MMU để ngăn ngừa mã độc ghi đè lên lệnh thực thi.
2. **Data Segment (`.data`) & BSS Segment (`.bss`):**
   - `.data`: Chứa các biến toàn cục (*global variables*) và biến tĩnh (*static variables*) đã được khởi tạo giá trị khác 0 tại thời điểm biên dịch.
   - `.bss` (*Block Started by Symbol*): Chứa các biến tĩnh/toàn cục chưa khởi tạo hoặc khởi tạo bằng 0. Khi nạp tiến trình, kernel chỉ cần cấp phát kích thước mà không tốn dung lượng lưu trữ trên đĩa cứng, sau đó ánh xạ về các trang bộ nhớ vật lý được xóa trắng về 0.
3. **Heap Segment:** Bộ nhớ động được điều phối bởi Trình quản lý bộ nhớ (*Memory Allocator*) của C/C++ runtime. Vùng nhớ này tăng trưởng theo chiều địa chỉ tăng dần thông qua các lời gọi hệ thống `brk()` hoặc `mmap()`.

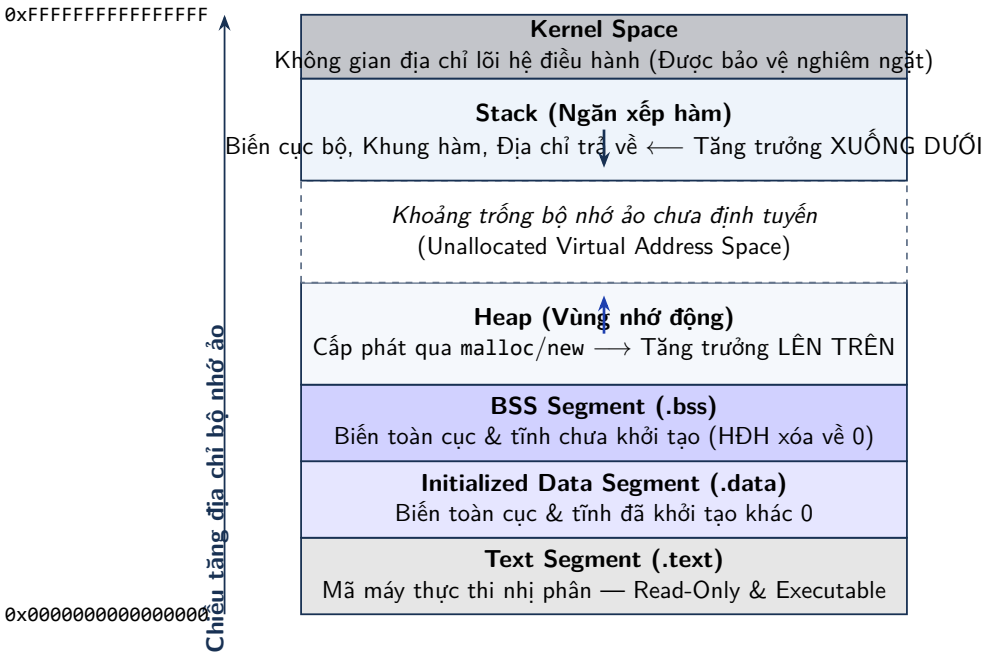


Figure 1.1: Kiến trúc phân tầng bộ nhớ ảo của một tiến trình người dùng trong kiến trúc x86-64.

4. **Stack Segment:** Vùng nhớ quản lý các khung hàm (*Stack Frames*). Khi một hàm được gọi, một khung ngăn xếp mới được cấp phát; khi hàm trả về, đỉnh ngăn xếp tự động co rút thông qua con trỏ thanh ghi `RSP`. Ngăn xếp tăng trưởng từ địa chỉ cao xuống địa chỉ thấp.[a]

[a] Nguồn: OOP1-C to C++-New.pdf - Trang 9-18: Process memory layout & Call stack architecture

1.3 Quản Lý Bộ Nhớ Động: So Sánh Đối Kháng malloc/free và new/delete

Sự khác biệt cốt tử giữa việc quản lý bộ nhớ trong C và C++ nằm ở sự hiểu biết về **Vòng đời của một Thực thể (Object Lifecycle)**.

```cpp
#include <iostream>
#include <cstdlib>

class SocketConnection {
private:
    int socketFd;
    char* networkBuffer;
public:
    SocketConnection(int port) : socketFd(port), networkBuffer(new char[1024]) {
        std::cout << "[Hardware] Socket initialized on port " << socketFd << "\n";
    }

    ~SocketConnection() {
        delete[] networkBuffer;
```

```
        std::cout << "[Hardware] Socket closed and buffer destroyed\n";
    }
};

void architecturalFailure() {
    // 1. C-STYLE ALLOCATION:
    // malloc() ONLY allocates raw bytes on Heap. IT NEVER CALLS CONSTRUCTOR!
    SocketConnection* pC = (SocketConnection*)malloc(sizeof(SocketConnection));

    // free() ONLY releases raw bytes. IT NEVER CALLS DESTRUCTOR!
    // Result: networkBuffer is leaked forever in RAM!
    free(pC);

    // 2. C++ PROPER ALLOCATION:
    // Operator new allocates memory AND executes SocketConnection::
    SocketConnection(8080)
    SocketConnection* pCpp = new SocketConnection(8080);

    // Operator delete executes ~SocketConnection() AND releases memory
    delete pCpp;
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Bản Chất Của Heap Cookie Và Hiểm Họa Trộn Lẫn new[

/ delete] Khi toán tử `new[]` cấp phát một mảng đối tượng:

```
SocketConnection* array = new SocketConnection[10];
```

Trình biên dịch thực hiện các thao tác mức máy sau:

1. **Tính toán kích thước thực tế:** Bộ nhớ cần cấp phát không chỉ là $10 \times \text{sizeof(SocketConnection)}$, mà được cộng thêm một phần phụ trội gọi là **Heap Cookie** (thường là 8 bytes trên kiến trúc 64-bit):
$$\text{Total Bytes} = \text{sizeof(size_t)} + 10 \times \text{sizeof(SocketConnection)}$$
2. **Ghi nhận số lượng phần tử:** Trình biên dịch ghi số nguyên `10` vào 8 bytes đầu tiên của khối bộ nhớ được cấp phát trên Heap.
3. **Dịch chuyển con trỏ:** Con trỏ trả về cho chương trình `array` được dịch chuyển qua khỏi Heap Cookie (tức trỏ vào phần tử đầu tiên).
4. **Khởi tạo mảng:** Trình biên dịch sinh vòng lặp gọi Constructor 10 lần liên tiếp.

Hậu quả khi gọi sai toán tử (`delete array` thay vì `delete[] array`): Trình biên dịch sẽ hiểu rằng `array` chỉ trỏ tới **MỘT đối tượng duy nhất**. Nó chỉ gọi Destructor đúng 1 lần cho phần tử đầu tiên, bỏ qua 9 phần tử còn lại (gây rò rỉ tài nguyên nội bộ nghiêm trọng). Sau đó, nó truyền địa chỉ `array` cho hàm thu hồi bộ nhớ của OS.
Vì hàm thu hồi bộ nhớ kỳ vọng địa chỉ bắt đầu của khối phân bổ gốc (chứa Heap Cookie), việc truyền địa chỉ bị lệch 8 bytes sẽ làm hỏng cấu trúc siêu dữ liệu của Heap (**Heap Corruption**), dẫn tới lỗi sập tiến trình ngay lập tức![a]

[a] Nguồn: OOP1-C to C++-New.pdf - Trang 19-27: So sánh new/delete và malloc/free, Cấu trúc Heap Cookie

1.4 Con Trỏ, Tham Chiếu và Cuộc Cách Mạng Move Semantics (C++11)

1.4.1 Bản Chất Dưới Gầm Máy: Tham Chiếu Lvalue (Lvalue Reference)

Trong ngôn ngữ C, mọi tham số truyền vào hàm mặc định là truyền theo giá trị (*Pass-by-value*). Để hàm có thể biến đổi giá trị bên ngoài, lập trình viên buộc phải sử dụng con trỏ (*Pointers*).

C++ giới thiệu khái niệm **Tham chiếu (Reference)**. Về mặt ngữ pháp, tham chiếu đóng vai trò là một bí danh (*Alias*) cho một đối tượng đã tồn tại:

```
int x = 42;
int& ref = x; // ref is an alias of x
ref = 100;    // x becomes 100
```

Tiên Đề / Định Nghĩa Pháp Lập: Bản Chất Hợp Ngữ Của Tham Chiếu Lvalue

Dưới góc nhìn kiến trúc máy tính, **Tham chiếu lvalue thực chất là một Con trỏ Hằng ngầm định (Constant Pointer)**:

$$\texttt{T\& ref = x;} \iff \texttt{T* const ref = \&x;}$$

Trình biên dịch sinh mã máy hợp ngữ hoàn toàn tương đồng: Nó lưu trữ địa chỉ của `x` vào một thanh ghi hoặc ô nhớ 8 bytes, và mỗi khi biểu thức sử dụng `ref`, trình biên dịch tự động chèn lệnh giải tham chiếu (*Dereferencing*):

```
; ref = 100
mov rax, QWORD PTR [rbp-8]    ; Load memory address of x stored in reference
mov DWORD PTR [rax], 100       ; Write integer 100 directly to x memory
    location
```

Điểm khác biệt tối hậu: Tham chiếu bắt buộc phải được khởi tạo ngay khi khai báo, không bao giờ được phép trỏ vào `nullptr`, và không thể trỏ sang đối tượng khác sau khi đã gắn kết.[a]

[a] Nguồn: OOP1-C to C++-New.pdf - Trang 28-35: Con trỏ và tham chiếu dưới tầng kiến trúc

1.4.2 Hệ Thống Phân Loại Giá Trị (Value Categories) Trong C++11

Trước C++11, các biểu thức chỉ được phân loại đơn giản thành **lvalue** (những gì có thể đứng bên trái phép gán) và **rvalue** (những gì chỉ có thể đứng bên phải phép gán). Chuẩn C++11 đã định nghĩa lại toàn diện lý thuyết biểu thức dựa trên hai đặc tính độc lập:

1. **Has Identity (Có định danh bộ nhớ):** Chương trình có thể xác định địa chỉ

bộ nhớ cụ thể (`&expr`) của đối tượng.

2. **Can be Moved from (Có thể di dời tài nguyên):** Trạng thái nội tại của đối tượng có thể được "đánh cắp" một cách an toàn mà không ảnh hưởng tới tính toàn vẹn của logic chương trình.

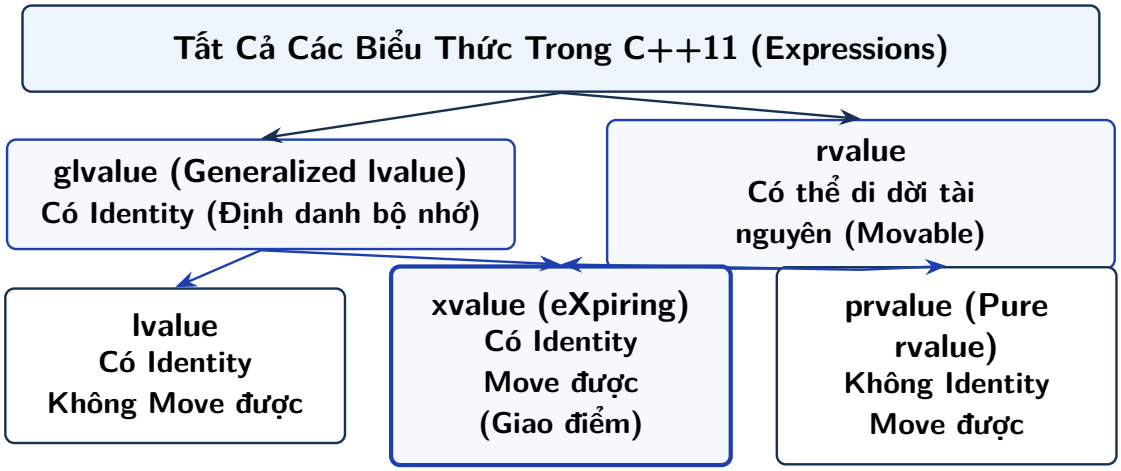


Figure 1.2: Cây phân loại giá trị chuẩn mực quốc tế ISO/IEC 14882:2011 (Value Categories).

- **lvalue (Locator Value):** Đại diện cho một đối tượng có vị trí bộ nhớ xác định và có tên (ví dụ: tên biến, hàm trả về tham chiếu lvalue `T&`).
- **prvalue (Pure Rvalue):** Đại diện cho các giá trị tạm thời tính toán tức thời hoặc hằng số nguyên thủy (ví dụ: số `42`, kết quả biểu thức `a + b`, hàm trả về theo giá trị `T`).
- **xvalue (eXpiring Value):** Đại diện cho một đối tượng có danh tính bộ nhớ nhưng vòng đời của nó chuẩn bị chấm dứt, cho phép các hàm khác đánh cắp tài nguyên của nó (ví dụ: kết quả của `std::move(var)`).

1.4.3 Tham Chiếu Rvalue (`T&&`) và Ngữ Nghĩa Di Chuyển (Move Semantics)

Trong C++98, việc truyền các đối tượng lớn (như chuỗi ký tự, mảng động, ma trận) ra khỏi hàm luôn đòi hỏi một phép sao chép sâu (*Deep Copy*) vô cùng đắt đỏ. Chuẩn C++11 giải quyết triệt để vấn đề này thông qua **Tham chiếu Rvalue (`T&&`)**.

```cpp
#include <iostream>
#include <utility>

class HeavyBuffer {
public:
    size_t size;
    int* data;

    // 1. Resource Allocating Constructor
    HeavyBuffer(size_t s) : size(s), data(new int[s]) {}

    // 2. Destructor
    ~HeavyBuffer() { delete[] data; }
```

```
14
15      // 3. Classical Copy Constructor (Deep Copy - O(N))
16      HeavyBuffer(const HeavyBuffer& other) : size(other.size), data(new int[other.
        size]) {
17          for (size_t i = 0; i < size; ++i) data[i] = other.data[i];
18      }
19
20      // 4. C++11 Move Constructor (Resource Theft - O(1))
21      HeavyBuffer(HeavyBuffer&& other) noexcept : size(other.size), data(other.data)
         {
22          // STEAL THE RESOURCE AND NEUTRALIZE SOURCE
23          other.data = nullptr;
24          other.size = 0;
25      }
26  };
```

Tiên Đề / Định Nghĩa Pháp Lập: Bản Chất Thực Sự Của `std::move`

Một trong những hiểu lầm phổ biến nhất của các kỹ sư là cho rằng hàm `std::move()` thực hiện việc di chuyển dữ liệu trong bộ nhớ RAM.

Sự thật kỹ thuật: `std::move()` hoàn toàn **KHÔNG DI CHUYỂN BẤT KỲ BYTE DỮ LIỆU NÀO** và **KHÔNG SINH RA BẤT KỲ LỆNH MÃ MÁY NÀO** tại thời điểm thực thi!

Bản chất của `std::move()` là một hàm khuôn mẫu thực hiện phép **Ép kiểu tĩnh không điều kiện** (*Unconditional Static Cast*) biến một biểu thức kiểu lvalue thành một biểu thức kiểu rvalue (chính xác là xvalue):

```
1  template <typename T>
2  constexpr typename std::remove_reference<T>::type&& move(T&& arg) noexcept {
3      return static_cast<typename std::remove_reference<T>::type&&>(arg);
4  }
```

Phép ép kiểu này cho phép trình biên dịch lựa chọn nạp chồng **Move Constructor** hoặc **Move Assignment Operator** thay vì Copy Constructor![a]

[a] Nguồn: OOP1-C to C++-New.pdf - Trang 36-45: C++11 Move Semantics, std::move và rvalue references

1.4.4 Quy Tắc Sụp Đổ Tham Chiếu (Reference Collapsing Rules)

Khi viết các hàm khuôn mẫu nhận tham số vạn năng (*Universal / Forwarding References*) có dạng `T&&`, trình biên dịch C++ áp dụng bốn quy tắc sụp đổ tham chiếu để xác định kiểu thực tế:

1. $\& + \& \longrightarrow \&$
2. $\& + \&\& \longrightarrow \&$
3. $\&\& + \& \longrightarrow \&$
4. $\&\& + \&\& \longrightarrow \&\&$

Dựa trên bốn quy tắc này, hàm `std::forward<T>()` được hiện thực hóa để bảo toàn trọn vẹn danh mục giá trị gốc (*Perfect Forwarding*): Nếu đối số ban đầu là lvalue, nó được chuyển tiếp dưới dạng lvalue; nếu là rvalue, nó được chuyển tiếp dưới dạng rvalue!

1.5 Hệ Thống Ép Kiểu An Toàn Chuẩn C++11

Ngôn ngữ C sử dụng cú pháp ép kiểu thô `(Type)expression`. Cách làm này bị xem là cực kỳ nguy hiểm trong công nghệ phần mềm hiện đại vì nó gộp chung mọi dạng biến đổi: chuyển đổi số học, xóa bỏ thuộc tính hằng (`const`), và tái thông dịch con trỏ mà không có sự kiểm tra tính hợp lệ của hệ thống kiểu.

C++ thiết lập bộ tứ toán tử ép kiểu tường minh với ngữ nghĩa chuyên biệt hóa:

1. **`static_cast<TargetType>(expr)`:**
   - Thực hiện các chuyển đổi kiểu hợp lệ tại thời điểm biên dịch: chuyển đổi số học (giữa `int`, `float`, `double`), gọi các hàm tạo đơn tham số (*Converting Constructors*), hoặc ép kiểu ngược từ `void*` về con trỏ đích đã biết.
   - Không thể dùng để xóa bỏ thuộc tính `const`.
2. **`const_cast<TargetType>(expr)`:**
   - Dùng duy nhất cho mục đích: Gắn thêm hoặc loại bỏ thuộc tính `const` hoặc `volatile` khỏi con trỏ hoặc tham chiếu.
   - *Cảnh báo an ninh:* Việc dùng `const_cast` để ghi dữ liệu vào một biến vốn được định nghĩa ban đầu là hằng số (`const`) nằm trong phân vùng bộ nhớ chỉ đọc (`.rodata`) sẽ dẫn tới lỗi sập tiến trình (**Segmentation Fault**).
3. **`reinterpret_cast<TargetType>(expr)`:**
   - Chỉ thị cho trình biên dịch: *"Hãy tái thông dịch chuỗi bit nhị phân của con trỏ này thành kiểu con trỏ khác mà không thực hiện bất kỳ phép biến đổi dữ liệu nào".*
   - Dùng để giao tiếp trực tiếp với phần cứng, driver, hoặc thao tác trên địa chỉ mạng nhị phân. Không an toàn và phụ thuộc chặt chẽ vào kiến trúc phần cứng (*Non-portable*).
4. **`dynamic_cast<TargetType>(expr)`:**
   - Thực hiện phép ép kiểu xuôi ngược trong cây phái sinh (*Downcasting*) tại **thời điểm thực thi (Runtime)**.
   - **Yêu cầu kỹ thuật bắt buộc:** Lớp cơ sở phải có **ít nhất một hàm ảo (`virtual`)** để trình biên dịch tạo thông tin nhận dạng kiểu động tại thời điểm chạy (**Run-Time Type Information - RTTI**).
   - Nếu ép kiểu con trỏ thất bại, nó trả về `nullptr`. Nếu ép kiểu tham chiếu thất bại, nó ném ngoại lệ `std::bad_cast`.[1]

[1]Nguồn: OOP1-C to C++-New.pdf - Trang 46-55: Hệ thống ép kiểu an toàn chuẩn C++

CHƯƠNG 2

Lớp & Đối tượng (Classes & Objects)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa Pháp Lập: Lớp (Class) và Đối Tượng (Object)

Lớp (Class) là một khuôn mẫu trừu tượng do người dùng định nghĩa, đóng vai trò là một kiểu dữ liệu mới (*User-Defined Type*), quy định cấu trúc bộ nhớ và các phép toán hợp lệ trên kiểu dữ liệu đó.
Đối tượng (Object) là một thực thể vật lý cụ thể (*Instance*) được cấp phát trong không gian bộ nhớ (trên Stack hoặc Heap), chiếm một vùng kích thước xác định và sở hữu trạng thái dữ liệu độc lập.[a]

[a]Nguồn: OOP2-introduction_new.pdf - Trang 4-12: Khái niệm lớp, đối tượng và đóng gói

2.1 Tính Đóng Gói (Encapsulation) & Bộ Chỉ Định Quyền Truy Cập

2.1.1 Cơ Chế Phân Quyền Truy Cập

Tính đóng gói (*Encapsulation*) trong C++ thực hiện việc che giấu cấu trúc dữ liệu nội tại và bảo vệ trạng thái bất biến (*Class Invariants*) của đối tượng thông qua ba từ khóa chỉ định phạm vi truy cập:

- **`public`:** Các thành viên có thể được truy cập tự do từ bất kỳ điểm nào trong chương trình thông qua biến đối tượng, con trỏ hoặc tham chiếu.
- **`private`:** Các thành viên bị khóa hoàn toàn; chỉ có thể được truy cập bởi các hàm thành viên của chính lớp đó hoặc bởi các hàm/lớp được cấp quyền bạn bè (`friend`).
- **`protected`:** Tương tự như `private`, nhưng cho phép các lớp con cháu trong cây

phái sinh (*Derived Classes*) có quyền truy cập trực tiếp.

Sự khác biệt cốt tử giữa `class` và `struct`: Trong C++, sự khác biệt duy nhất giữa `class` và `struct` nằm ở **Quyền truy cập mặc định**:

- Trong `struct`, mọi thành viên và mối quan hệ kế thừa mặc định là **`public`**.
- Trong `class`, mọi thành viên và mối quan hệ kế thừa mặc định là **`private`**.

2.1.2 Khảo Sát Bộ Nhớ: Sự Căn Chỉnh Dữ Liệu (Memory Alignment & Padding)

Kích thước của một đối tượng trong RAM không đơn thuần là tổng kích thước của các biến thành viên:

```
struct UnalignedStruct {
    char a;     // 1 byte
    // 3 bytes padding do kien truc 32-bit/64-bit yeu cau can le 4 bytes
    int b;      // 4 bytes
    char c;     // 1 byte
    // 7 bytes padding do kien truc 64-bit yeu cau can le 8 bytes
    double d;   // 8 bytes
}; // sizeof(UnalignedStruct) = 24 bytes (thay vi 1 + 4 + 1 + 8 = 14 bytes)!

struct OptimizedStruct {
    double d;   // 8 bytes (Offset 0)
    int b;      // 4 bytes (Offset 8)
    char a;     // 1 byte  (Offset 12)
    char c;     // 1 byte  (Offset 13)
    // 2 bytes padding de tong kich thuoc chia het cho 8
}; // sizeof(OptimizedStruct) = 16 bytes! Tiet kiem 33% bo nho RAM!
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Nguyên Lý Căn Lề Bộ Nhớ Phần Cứng (Hardware Bus Alignment)

Tại sao CPU không đọc dữ liệu tại bất kỳ địa chỉ nào mà đòi hỏi căn lề (*Data Alignment*)?

1. **Đường truyền Bus dữ liệu:** Bộ vi xử lý hiện đại (x86-64) đọc dữ liệu từ RAM vào CPU Cache theo từng khối 32-bit, 64-bit, hoặc Cache Line 64 bytes.
2. **Chi phí truy cập lệch lề (Misaligned Access):** Nếu một biến số thực 8 bytes (`double`) nằm vắt ngang qua ranh giới của hai khối 8 bytes, CPU buộc phải thực hiện **HAI chu kỳ đọc bộ nhớ riêng biệt**, sau đó dùng các lệnh dịch bit (`shl`, `shr`) và ghép bit (`or`) để tái tạo lại giá trị.
3. Trình biên dịch C++ tự động chèn các byte đệm rác (**Padding Bytes**) để tối ưu hóa tốc độ truy xuất của CPU, ưu tiên hiệu năng thực thi hơn việc tiết kiệm vài byte bộ nhớ vật lý.[a]

[a] Nguồn: OOP3-Class&Objec_new.pdf - Trang 5-15: Cấu trúc lớp và tổ chức dữ liệu trong bộ nhớ

2.2 Bản Chất Con Trỏ this Dưới Tầng RAM & ABI Gọi Hàm

Một đối tượng khi được tạo ra trong RAM chỉ chiếm dung lượng bằng tổng các biến thành viên dữ liệu (cộng padding và con trỏ ảo nếu có). **Các hàm thành viên không chiếm bất kỳ byte nào trong không gian nhớ của đối tượng**! Mã máy của các hàm nằm duy nhất một bản trong phân vùng `.text`.

Làm thế nào một hàm thành viên biết nó đang thao tác trên dữ liệu của đối tượng nào? Câu trả lời là **Con trỏ ngầm định `this`**.

```
class RegisterFile {
private:
    int value;
public:
    void setValue(int v) {
        this->value = v; // this la con tro hang tro toi chinh doi tuong
    }
};
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Giao Diện Nhị Phân Ứng Dụng (ABI) Và Thanh Ghi Truyền Con Trỏ this

Khi lập trình viên viết dòng lệnh gọi hàm: `reg.setValue(42);`, trình biên dịch biến đổi lời gọi này thành một hàm dạng thủ tục mức thấp:

```
RegisterFile::setValue(&reg, 42);
```

Quy ước gọi hàm (Calling Conventions):

- **Trên kiến trúc 32-bit x86 (`__thiscall`):** Con trỏ `this` được truyền trực tiếp thông qua thanh ghi chuyên dụng **`ECX`**, trong khi các tham số còn lại được đẩy vào Stack từ phải qua trái.
- **Trên kiến trúc 64-bit (System V AMD64 ABI - Linux/macOS):** Con trỏ `this` luôn được nạp vào thanh ghi tham số đầu tiên: **`RDI`**. Tham số thứ hai được nạp vào **`RSI`**.
- **Trên kiến trúc 64-bit (Microsoft x64 ABI - Windows):** Con trỏ `this` được nạp vào thanh ghi **`RCX`**. Tham số thứ hai được nạp vào **`RDX`**.

Mã máy hợp ngữ được sinh ra minh họa cơ chế này:

```
lea rdi, [rbp-16]       ; Load dia chi doi tuong 'reg' vao thanh ghi RDI (
    this pointer)
mov esi, 42             ; Load tham so 42 vao thanh ghi RSI
call RegisterFile::setValue ; Nhay toi ham thanh vien trong .text
```

Thuộc tính bất biến: `this` là một con trỏ hằng có kiểu `ClassName* const`. Đối với các phương thức hằng (`const` method), nó có kiểu `const ClassName* const`.[a]

[a] Nguồn: OOP3-Class&Objec_new.pdf - Trang 16-25: Con trỏ this và cơ chế truyền tham số ngầm định

2.3 Vòng Đời Khởi Tạo Đối Tượng (Constructors Architecture)

Hàm dựng (**Constructor**) là hàm thành viên đặc biệt chịu trách nhiệm thiết lập trạng thái ban đầu của đối tượng và thiết lập các điều kiện bất biến (*Class Invariants*). Constructor không có kiểu trả về và mang tên trùng khớp với tên lớp.

2.3.1 Member Initializer List vs Phép Gán Trong Thân Constructor

```
class PipelineBlock {
private:
    std::string blockName;
    const int blockId;      // Const field: Bat buoc phai khoi tao bang danh sach
    int& systemRef;         // Reference field: Bat buoc phai khoi tao bang danh
    sach

public:
    // PHUONG PHAP CHUAN MUC (Member Initializer List):
    PipelineBlock(const std::string& name, int id, int& ref)
        : blockName(name), blockId(id), systemRef(ref) {
        // Truong du lieu duoc khoi tao truc tiep dung Constructor tuong ung!
    }

    // PHUONG PHAP PHAM QUY (Assignment inside body):
    /*
    PipelineBlock(const std::string& name, int id, int& ref) {
        blockName = name; // LANG PHI: Goi Default Ctor roi moi goi Assignment!
        // blockId = id;  // LOI BIEN DICH: Khong the gan cho const!
        // systemRef = ref; // LOI BIEN DICH: Khong the gan cho reference!
    }
    */
};
```

Tiên Đề / Định Nghĩa Pháp Lập: Bản Chất Của Danh Sách Khởi Tạo Thành Viên (Member Initializer List)

Khi một đối tượng được khởi tạo, tiến trình cấp phát diễn ra theo hai giai đoạn độc lập:

1. **Giai đoạn 1: Khởi tạo các trường thành viên (Initialization Phase):** Xảy ra tại danh sách khởi tạo `: var1(val1), var2(val2)`. Mỗi trường được gọi trực tiếp hàm tạo tương ứng.
2. **Giai đoạn 2: Thực thi phần thân hàm tạo (Execution Phase):** Các dòng lệnh bên trong cặp dấu ngoặc nhọn `{ }` được thực thi. Nếu tại đây có phép gán `var1 = val1;`, trường đó đã phải trải qua việc khởi tạo mặc định từ trước, sau đó bị ghi đè bởi toán tử gán, gây lãng phí ít nhất 50% hiệu năng xử lý!

Hành Vi Không Xác Định (Undefined Behavior): Bẫy Thứ Tự Khởi Tạo Thành Viên (Initialization Order Trap)

Xét đoạn mã nguồn sau:

```
class MemoryController {
private:
    size_t bufferSize;
    int* buffer;
public:
    // Canh bao chet nguoi: buffer duoc viet truoc bufferSize trong danh
    sach!
    MemoryController(size_t size)
        : buffer(new int[bufferSize]), bufferSize(size) {}
};
```

Quy tắc chuẩn quốc tế ISO/IEC 14882: Thứ tự khởi tạo các trường thành viên của một lớp **HOÀN TOÀN DỰA VÀO THỨ TỰ KHAI BÁO TRONG ĐỊNH NGHĨA LỚP**, hoàn toàn bất chấp thứ tự được liệt kê trong Member Initializer List!
Trong ví dụ trên, biến `bufferSize` được khai báo **TRƯỚC** biến `buffer`. Do đó, `bufferSize` sẽ được khởi tạo trước! Tuy nhiên, nếu một lập trình viên vô tình đảo thứ tự khai báo: `int* buffer;` đứng trước `size_t bufferSize;`, thì `buffer` sẽ được cấp phát trước khi `bufferSize` có giá trị (tức `bufferSize` đang giữ rác bộ nhớ). Kết quả là hệ thống cố gắng cấp phát một lượng RAM ngẫu nhiên khổng lồ, gây sập chương trình ngay lập tức (**Segmentation Fault**)![a]

[a]Nguồn: OOP3-Class&Objec_new.pdf - Trang 26-40: Vòng đời Constructor và thứ tự khởi tạo

2.3.2 Ngăn Chặn Ép Kiểu Ngầm Định Với Từ Khóa `explicit`

Trong C++, bất kỳ Constructor nào có thể được gọi với **MỘT tham số duy nhất** đều mặc định đóng vai trò là một hàm chuyển đổi kiểu ngầm định (*Converting Constructor*):

```
class BufferPool {
public:
    // explicit ngan chan compiler tu dong ep kieu int thanh BufferPool
    explicit BufferPool(int capacity) { ... }
};

void allocate(const BufferPool& pool);

void test() {
    // BufferPool p = 100; // LOI BIEN DICH: explicit da chan ep kieu ngam dinh!
    BufferPool p(100);    // HOP LE: Khoi tao tuong minh
    allocate(BufferPool(50)); // HOP LE
}
```

2.4 Hàm Hủy (Destructor) & Triết Lý Thiết Kế RAII

Hàm hủy (**Destructor**) có cú pháp `~ClassName()`, không nhận tham số và không trả về giá trị. Destructor được hệ thống tự động kích hoạt khi đối tượng chuẩn bị kết thúc vòng đời.

Tiên Đề / Định Nghĩa Pháp Lập: Triết Lý Thiết Kế RAII (Resource Acquisition Is Initialization)

Được phát minh bởi Bjarne Stroustrup, **RAII** là mô thức thiết kế phần mềm quan trọng nhất của C++ hiện đại. Tư tưởng cốt lõi của RAII thiết lập định đề:

"Mọi tài nguyên hệ thống (Bộ nhớ RAM, File Descriptor, Socket Mạng, Mutex Lock) phải được gắn chặt vào Vòng đời của một Đối tượng Ngăn xếp (Stack-allocated Object)."

- **Cấp phát tài nguyên** được thực hiện trong Constructor.
- **Giải phóng tài nguyên** được ủy thác hoàn toàn tự động cho Destructor.

Nhờ cơ chế **Giải phóng Ngăn xếp (Stack Unwinding)**, cho dù hàm kết thúc bình thường qua lệnh `return` hay bị gián đoạn đột ngột bởi một ngoại lệ (`throw`), toàn bộ tài nguyên đều được dọn dẹp sạch sẽ 100%, loại trừ hoàn toàn nguy cơ rò rỉ tài nguyên (*Resource Leak*)![a]

[a]Nguồn: OOP3-Class&Objec_new.pdf - Trang 41-52: Destructor và triết lý quản trị tài nguyên RAII

2.5 Thành Viên Static, Const và Kỹ Thuật Hàm Bạn (Friend)

2.5.1 Thành Viên Tĩnh (Static Data & Methods)

- **Biến thành viên tĩnh (`static`):** Không thuộc về bất kỳ đối tượng cụ thể nào của lớp. Nó chỉ tồn tại **DUY NHẤT MỘT BẢN SAO** trong toàn bộ tiến trình, được lưu trữ tại phân vùng `.data` hoặc `.bss`. Kích thước của đối tượng hoàn toàn không chứa biến `static`.
- **Phương thức tĩnh (`static` methods):** Không nhận con trỏ ngầm định `this`. Do đó, phương thức tĩnh **không thể truy cập các biến thành viên phi tĩnh** và không thể gọi các phương thức phi tĩnh nếu không thông qua một đối tượng tường minh.

2.5.2 Từ Khóa `mutable` — Ngoại Lệ Trong Phương Thức Hằng

Một phương thức được đánh dấu `const` cam kết không làm thay đổi trạng thái quan sát được (*Bitwise Constness*) của đối tượng. Tuy nhiên, trong một số kịch bản kỹ thuật (như đồng bộ hóa luồng qua `std::mutex`, lưu bộ nhớ đệm cache, đếm tần suất

truy cập), ta cần biến đổi một số trường dữ liệu mà không làm thay đổi ngữ nghĩa logic của đối tượng. Từ khóa **`mutable`** cho phép một trường dữ liệu được phép sửa đổi ngay bên trong phương thức `const`:

```
class DataCache {
private:
    std::string cachedValue;
    mutable size_t accessCounter = 0; // Duoc phep bien doi trong const method

public:
    std::string readData() const {
        ++accessCounter; // HOP LE nho tu khoa mutable!
        return cachedValue;
    }
};
```

2.5.3 Quan Hệ Bạn Bè (Friendship Architecture)

Từ khóa `friend` cho phép một hàm tự do hoặc một lớp khác có quyền truy cập vào các trường `private` và `protected` của lớp định nghĩa.

Tiên Đề / Định Nghĩa Pháp Lập: Ba Định Đề Bất Biến Về Quan Hệ Bạn Bè (Friendship Postulates)

1. **Tính Không Đối Xứng (Not Symmetric):** Nếu lớp A khai báo lớp B là bạn, thì B có quyền xem trường ẩn của A, nhưng A hoàn toàn không có quyền xem trường ẩn của B.
2. **Tính Không Bắc Cầu (Not Transitive):** Nếu A là bạn của B, và B là bạn của C, thì A hoàn toàn không mặc nhiên là bạn của C.
3. **Tính Không Kế Thừa (Not Inherited):** Lớp con của một lớp bạn không được thừa hưởng quyền bạn bè từ lớp cha.[a]

[a] Nguồn: OOP3-Class&Objec_new.pdf - Trang 53-65: Static, Const, Mutable và Quan hệ Friend

CHƯƠNG 3

Quản trị Bộ nhớ & Nạp chồng Toán tử (Memory & Overloading)

Tiên Đề / Định Nghĩa Pháp Lập: Nguyên Lý Quản Trị Tài Nguyên Bộ Nhớ

Khả năng kiểm soát bộ nhớ RAM ở cấp độ byte là phẩm chất định nghĩa nên uy lực của C++. Tuy nhiên, tự do đi liền với trách nhiệm: Một kiến trúc phần mềm chuẩn mực phải đảm bảo hai điều kiện biên bất biến:

1. **Tính Toàn Vẹn Sở Hữu (Ownership Integrity):** Mỗi khối tài nguyên động trên Heap phải có một chủ sở hữu duy nhất hoặc một nhóm chủ sở hữu được kiểm soát nghiêm ngặt.
2. **Tính Bất Biến Về Vòng Đời (Lifecycle Invariant):** Tài nguyên phải được giải phóng đúng một lần duy nhất (*Exactly-once Destruction*) ngay khi chủ sở hữu cuối cùng kết thúc vòng đời.[a]

[a] Nguồn: OOP4-OperatorOverloading_new.pdf - Trang 2-10: Quản trị tài nguyên và nạp chồng toán tử

3.1 Sao Chép Nông (Shallow Copy) vs Sao Chép Sâu (Deep Copy)

Khi lập trình viên không khai báo Copy Constructor hoặc Copy Assignment Operator, trình biên dịch sẽ tự động sinh ra một phiên bản mặc định thực hiện **Sao chép nông (Shallow Copy / Memberwise Copy)**.

3.1.1 Cơ Chế Phá Hủy Bộ Nhớ Của Shallow Copy (Double-Free Disaster)

Xét một lớp quản lý một mảng động trên Heap:

```
1 class RawArray {
```

```
public:
    int* data;
    size_t size;

    RawArray(size_t s) : size(s), data(new int[s]) {}
    ~RawArray() { delete[] data; }
    // No custom Copy Constructor -> Compiler generates shallow copy!
};

void catastrophicExecution() {
    RawArray a(100);
    RawArray b = a; // SHALLOW COPY: b.data gets the exact pointer value of a.data
    !
} // Scope exits: SYSTEM CRASH HAPPENS HERE!
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Giải Phẫu Thảm Họa Double-Free Dưới Tầng Bộ Nhớ

Hãy khảo sát chính xác trạng thái của thanh ghi và bộ nhớ trong hàm `catastrophicExecution()`:

1. Khi `b = a` được thực thi, trình biên dịch chỉ đơn thuần sao chép giá trị số nguyên của con trỏ: `b.data = a.data`.
2. **Hiện tượng Con trỏ trùng lặp (Pointer Aliasing):** Cả `a.data` và `b.data` đều cùng trỏ vào **DUY NHẤT MỘT ĐỊA CHỈ HEAP** (ví dụ `0x00A01234`).
3. **Giai đoạn Hủy (Destruction):**
   - Theo quy tắc LIFO của Stack, đối tượng `b` bị hủy trước. Destructor `~RawArray()` của `b` thực thi lệnh `delete[] b.data`, giải phóng khối bộ nhớ tại `0x00A01234` về cho hệ điều hành. Vùng nhớ này bị đánh dấu là *Free Chunk*.
   - Ngay sau đó, đối tượng `a` bị hủy. Destructor của `a` tiếp tục gọi `delete[] a.data` trên cùng địa chỉ `0x00A01234`!
4. **Hậu quả hệ thống:** Việc giải phóng một vùng nhớ đã được giải phóng từ trước gọi là **Lỗi Double-Free**. C/C++ Memory Allocator lập tức phát hiện sự sai lệch trong danh sách liên kết của Heap và ném tín hiệu **`SIGABRT`** (hoặc `Heap Corruption Exception`), làm sập tiến trình ngay lập tức![a]

[a] Nguồn: OOP4-OperatorOverloading_new.pdf - Trang 11-18: Shallow copy vs Deep copy và lỗi Double-Free

3.1.2 Giải Pháp Chuẩn Mực: Sao Chép Sâu (Deep Copy)

Sao chép sâu đòi hỏi việc cấp phát một khối bộ nhớ mới hoàn toàn độc lập trên Heap, sau đó sao chép toàn bộ nội dung dữ liệu từ nguồn sang đích:

```
// Deep Copy Constructor
RawArray::RawArray(const RawArray& other)
    : size(other.size), data(new int[other.size]) {
    for (size_t i = 0; i < size; ++i) {
        data[i] = other.data[i];
    }
```

```
}
```

3.2 Tiến Hóa Bộ Quy Tắc Quản Trị Tài Nguyên: Rule of 3, 5, 0

3.2.1 Quy Tắc Bộ Ba Cổ Điển (The Rule of Three — C++98)

Tiên Đề / Định Nghĩa Pháp Lập: Quy Tắc Bộ Ba (The Rule of Three)

Nếu một lớp đòi hỏi phải hiện thực hóa tường minh một trong ba phương thức sau:

1. **Hàm Hủy (Destructor)**
2. **Hàm Tạo Sao Chép (Copy Constructor)**
3. **Toán Tử Gán Sao Chép (Copy Assignment Operator)**

thì lớp đó **HẦU NHƯ CHẮC CHẮN PHẢI HIỆN THỰC HÓA CẢ BA PHƯƠNG THỨC**! Sự hiện diện của một hàm hủy tường minh chứng minh rằng lớp đang nắm giữ một tài nguyên thô (Raw Resource); do đó các phương thức sao chép mặc định sẽ gây ra lỗi Shallow Copy.[a]

[a]Nguồn: OOP4-OperatorOverloading_new.pdf - Trang 19-25: Quy tắc Rule of Three trong C++98

3.2.2 Quy Tắc Bộ Năm Hiện Đại (The Rule of Five — C++11)

Sự xuất hiện của Ngữ nghĩa Di chuyển (Move Semantics) trong chuẩn C++11 mở rộng Rule of Three thành **Rule of Five**. Một lớp quản lý tài nguyên phải thiết lập 5 phương thức đặc biệt:

1. **Destructor:** `~Resource()`
2. **Copy Constructor:** `Resource(const Resource&)`
3. **Copy Assignment Operator:** `Resource& operator=(const Resource&)`
4. **Move Constructor:** `Resource(Resource&&) noexcept`
5. **Move Assignment Operator:** `Resource& operator=(Resource&&) noexcept`

3.2.3 Thành Ngữ Copy-and-Swap (The Copy-and-Swap Idiom)

Để hiện thực hóa cả Copy Assignment và Move Assignment một cách thanh lịch, an toàn ngoại lệ tuyệt đối mà không lặp lại mã nguồn, kỹ sư C++ sử dụng kỹ thuật **Copy-and-Swap**:

```
#include <utility>
#include <algorithm>

class ManagedBuffer {
```

```
private:
    size_t length;
    char* buffer;

public:
    // 1. Constructor
    ManagedBuffer(size_t len = 0)
        : length(len), buffer(len ? new char[len] : nullptr) {}

    // 2. Destructor
    ~ManagedBuffer() { delete[] buffer; }

    // 3. Copy Constructor (Deep Copy)
    ManagedBuffer(const ManagedBuffer& other)
        : length(other.length), buffer(other.length ? new char[other.length] :
    nullptr) {
        std::copy(other.buffer, other.buffer + length, buffer);
    }

    // 4. Move Constructor (Zero Overhead Theft)
    ManagedBuffer(ManagedBuffer&& other) noexcept
        : ManagedBuffer() { // Delegate to Default Ctor
        swap(*this, other);
    }

    // Friend swap function: No-fail guarantee
    friend void swap(ManagedBuffer& first, ManagedBuffer& second) noexcept {
        using std::swap;
        swap(first.length, second.length);
        swap(first.buffer, second.buffer);
    }

    // 5. UNIFIED ASSIGNMENT OPERATOR (Covers both Copy and Move!)
    // Parameter passed BY VALUE triggers Copy Ctor for lvalue or Move Ctor for
    rvalue!
    ManagedBuffer& operator=(ManagedBuffer other) noexcept {
        swap(*this, other); // Swap internal state with temporary copy
        return *this;
    } // temporary 'other' is destroyed here, cleanly freeing old resource!
};
```

3.2.4 Quy Tắc Số Không (The Rule of Zero — Modern C++)

Tiên Đề / Định Nghĩa Pháp Lập: Đỉnh Cao Thiết Kế: Quy Tắc Số Không (The Rule of Zero)

Trong kiến trúc hướng đối tượng C++ hiện đại, lập trình viên **KHÔNG NÊN TỰ VIẾT BẤT KỲ PHƯƠNG THỨC NÀO TRONG BỘ NĂM**. Thay vì quản lý con trỏ thô, hãy ủy thác toàn bộ việc quản lý tài nguyên cho các lớp thư viện chuẩn đã đạt chuẩn RAII như `std::unique_ptr`, `std::vector`, hoặc `std::string`. Khi đó, trình biên dịch sẽ tự động sinh ra toàn bộ các hàm tạo sao chép, di chuyển và hàm hủy với độ an toàn và hiệu năng tối ưu tuyệt đối![a]

[a]Nguồn: OOP4-OperatorOverloading_new.pdf - Trang 26-32: Rule of Five và Rule of Zero trong C++11

3.3 Nạp Chồng Toán Tử Toàn Diện (Operator Overloading Deep Dive)

C++ cho phép lập trình viên định nghĩa lại ngữ nghĩa của các toán tử ngôn ngữ đối với các kiểu dữ liệu tự định nghĩa, biến mã nguồn trở nên tự nhiên và biểu cảm như các biểu thức toán học.

3.3.1 Các Định Đề Bất Biến Về Toán Tử

Tiên Đề / Định Nghĩa Pháp Lập: Ba Giới Hạn Cốt Tử Của Ngôn Ngữ Về Nạp Chồng Toán Tử

1. **Bất Biến Về Độ Ưu Tiên & Kết Hợp:** Việc nạp chồng toán tử **tuyệt đối không thể làm thay đổi độ ưu tiên** (*Precedence*) hoặc **tính kết hợp** (*Associativity*) của toán tử (ví dụ: toán tử * luôn được thực thi trước toán tử +).
2. **Bất Biến Về Số Lượng Toán Hạng:** Không thể biến đổi toán tử hai ngôi thành một ngôi hoặc ngược lại.
3. **Cấm Khởi Tạo Toán Tử Mới:** Lập trình viên không thể tự chế tạo ra các ký tự toán tử mới không tồn tại trong ngữ pháp chuẩn C++ (như ** hoặc %%).

Năm toán tử CẤM TUYỆT ĐỐI không được nạp chồng trong C++:

1. Toán tử truy cập thành viên trực tiếp: `.`
2. Toán tử con trỏ tới thành viên trực tiếp: `.*`
3. Toán tử phân giải phạm vi: `::`
4. Toán tử điều kiện tam phân: `?:`
5. Toán tử kích thước: **`sizeof`** (và toán tử nhận dạng kiểu **`typeid`**).

3.3.2 Lựa Chọn Thiết Kế: Member Method vs Non-member Friend

- **Toán tử bắt buộc phải là Phương thức Thành viên (Member Functions):** Chuẩn C++ quy định 4 toán tử sau **bắt buộc 100% phải là member**:
  1. Toán tử gán: `operator=`
  2. Toán tử chỉ mục mảng: `operator[]`
  3. Toán tử gọi hàm: `operator()`

4. Toán tử truy cập gián tiếp con trỏ: `operator->`

- **Tại sao toán tử xuất/nhập luồng (<< và >>) BẮT BUỘC phải là Non-member Function?** Xét biểu thức: `std::cout << myObject;`. Toán hạng bên trái là `std::cout` (có kiểu `std::ostream&`). Nếu ta cài đặt dạng hàm thành viên, hàm đó bắt buộc phải nằm bên trong mã nguồn của lớp `std::ostream` thuộc thư viện chuẩn của trình biên dịch! Vì lập trình viên không thể can thiệp vào mã nguồn của `std::ostream`, ta bắt buộc phải nạp chồng `operator<<` dưới dạng **Hàm Tự Do Ngoài Lớp (Non-member Function)** và cấp quyền `friend` để truy cập trường ẩn:

```
    friend std::ostream& operator<<(std::ostream& os, const Complex& c) {
        os << "(" << c.real << " + " << c.imag << "i)";
        return os; // Tra ve ostream& de cho phep chain goi lien tiep
    }

```

3.3.3 Toán Tử Chỉ Mục `operator[]` & Toán Tử Gọi Hàm `operator()`

Để bảo đảm tính toàn vẹn *const-correctness*, toán tử chỉ mục luôn phải được viết thành **MỘT CẶP ĐỒNG BỘ**:

```
class SafeVector {
private:
    int* elements;
    size_t length;
public:
    // 1. Non-const version: Allows reading and writing
    int& operator[](size_t index) {
        return elements[index];
    }

    // 2. Const version: Allows reading from const instances
    const int& operator[](size_t index) const {
        return elements[index];
    }
};
```

3.3.4 Toán Tử Tăng Giảm: Tiền Tố (Prefix) vs Hậu Tố (Postfix)

```
class Counter {
private:
    int value = 0;
public:
    // 1. PREFIX (++c): Increments and returns reference to self
    Counter& operator++() {
        ++value;
        return *this; // Optimal performance: O(1), no temporary object
    }

    // 2. POSTFIX (c++): Takes dummy int parameter to differentiate
    Counter operator++(int) {
        Counter temp = *this; // MANDATORY: Must create a temporary copy!
```

```
        ++value;              // Increment actual state
        return temp;          // Return old value before increment
    } // Overhead: Triggers Copy Constructor
};
```

Hệ luận kỹ thuật: Trong các vòng lặp thuật toán, luôn ưu tiên sử dụng toán tử tiền tố (`++it`) thay vì hậu tố (`it++`) để triệt tiêu chi phí khởi tạo đối tượng tạm thời.[1]

3.4 Nạp Chồng Toán Tử new và delete & Kỹ Thuật Placement New

3.4.1 Nạp Chồng new/delete Cấp Lớp (Class-specific Allocators)

Các hệ thống giao dịch tần suất cao hoặc engine trò chơi thường không sử dụng bộ cấp phát tổng quát của hệ điều hành vì chi phí tìm kiếm vùng nhớ tự do quá lớn. C++ cho phép nạp chồng toán tử `new` và `delete` riêng cho từng lớp để lấy bộ nhớ từ một vùng nhớ được cấp phát tĩnh từ trước (*Memory Pool*):

```
class HighSpeedNode {
public:
    void* operator new(size_t bytes) {
        // Fast allocate from custom memory pool in O(1)
        return MemoryPool::allocateFast(bytes);
    }

    void operator delete(void* ptr) noexcept {
        // Return block back to pool in O(1)
        MemoryPool::deallocateFast(ptr);
    }
};
```

3.4.2 Kỹ Thuật Khởi Tạo Tại Chỗ (Placement New)

Placement New cho phép lập trình viên khởi tạo một đối tượng trực tiếp trên một địa chỉ bộ nhớ đã được phân bổ sẵn mà **hoàn toàn không thực hiện bất kỳ lệnh xin cấp phát RAM nào**:

```
#include <new> // Required for placement new

alignas(SocketConnection) char arenaBuffer[sizeof(SocketConnection)];

void testPlacementNew() {
    // 1. Construct object directly on arenaBuffer:
    SocketConnection* pConn = new (arenaBuffer) SocketConnection(9090);

    // 2. DESTRUCTION RULE: STRICTLY FORBIDDEN TO CALL delete pConn!
    // Since arenaBuffer is on Stack/Static, calling delete triggers a crash!

```

[1] Nguồn: OOP4-OperatorOverloading_new.pdf - Trang 33-50: Kỹ thuật nạp chồng toán tử nâng cao

```
    // 3. MUST MANUALLY INVOKE DESTRUCTOR EXPLICITLY:
    pConn->~SocketConnection();
}
```

3.5 Kiến Trúc Con Trỏ Thông Minh Chuẩn C++11 (Smart Pointers Architecture)

Để chấm dứt kỷ nguyên rò rỉ bộ nhớ của con trỏ thô (*Raw Pointers*), chuẩn C++11 giới thiệu bộ ba con trỏ thông minh trong thư viện `<memory>`.

3.5.1 `std::unique_ptr<T>` — Quyền Sở Hữu Độc Quyền (Exclusive Ownership)

- **Đặc tính kiến trúc:** Quản lý một đối tượng duy nhất. Không thể sao chép (`Copy Constructor = delete`), chỉ có thể di dời quyền sở hữu thông qua Move Semantics (`std::move`).
- **Hiệu năng:** Kích thước bộ nhớ của `std::unique_ptr<T>` bằng chính xác **MỘT con trỏ thô** (`sizeof = 8 bytes` trên 64-bit). Mọi thao tác truy xuất đều được inline hóa 100%, đạt hiệu năng ngang bằng con trỏ thô (**Zero Runtime Overhead**)!

3.5.2 `std::shared_ptr<T>` — Quyền Sở Hữu Chia Sẻ (Shared Ownership)

Nhiều con trỏ `shared_ptr` có thể cùng trỏ tới một đối tượng. Đối tượng sẽ được tự động giải phóng khi con trỏ `shared_ptr` cuối cùng bị hủy.

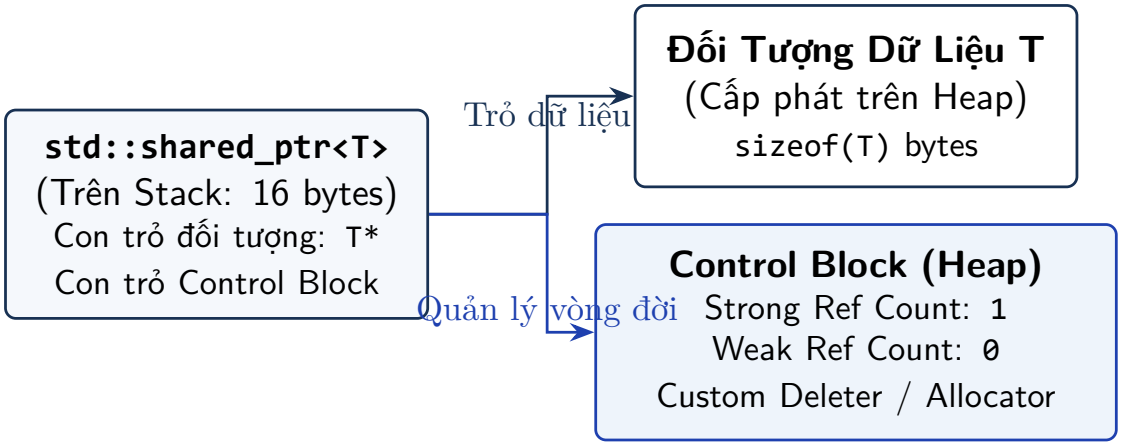


Figure 3.1: Kiến trúc bộ nhớ hai con trỏ và Khối Điều Khiển (Control Block) của `std::shared_ptr`.

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Kiến Trúc Khối Điều Khiển (Control Block) Của `std::shared_ptr`

Kích thước của một `std::shared_ptr` gấp đôi con trỏ thô (16 bytes trên nền tảng 64-bit), bao gồm:

1. **Con trỏ thứ nhất (8 bytes):** Trỏ thẳng tới đối tượng dữ liệu thực tế (`T*`).
2. **Con trỏ thứ hai (8 bytes):** Trỏ tới một cấu trúc cấp phát động độc lập gọi là **Khối Điều Khiển (Control Block)**.

Cấu trúc nội tại của Control Block:

- **Strong Reference Count (Bộ đếm tham chiếu mạnh):** Đếm số lượng `shared_ptr` đang cùng sở hữu đối tượng. Mọi thao tác tăng/giảm biến đếm này đều sử dụng các chỉ lệnh khóa luồng mức máy (**Atomic Instructions** như `lock xadd` trên x86) để đảm bảo an toàn luồng (*Thread-safety*).
- **Weak Reference Count (Bộ đếm tham chiếu yếu):** Đếm số lượng `weak_ptr` đang quan sát đối tượng.
- **Custom Deleter & Allocator:** Con trỏ hàm giải phóng tùy biến.

Ưu thế vượt trội của `std::make_shared<T>()`:

- Nếu dùng `std::shared_ptr<T>(new T())`: Hệ điều hành phải thực hiện **HAI lần cấp phát bộ nhớ động riêng rẽ** (một lần cho `T`, một lần cho Control Block), làm phân mảnh bộ nhớ và suy giảm hiệu năng CPU Cache.
- Nếu dùng `std::make_shared<T>()`: Hệ điều hành cấp phát **DUY NHẤT MỘT KHỐI RAM LIÊN TỤC** chứa cả Control Block và Đối tượng dữ liệu. Tối ưu hóa tuyệt đối tính cục bộ bộ nhớ đệm (**Cache Locality**)![a]

[a] Nguồn: OOP4-OperatorOverloading_new.pdf - Trang 51-68: Kiến trúc Smart Pointers và Control Block

3.5.3 `std::weak_ptr<T>` — Bẻ Gãy Vòng Lặp Tham Chiếu (Circular Reference)

Nếu hai đối tượng `A` và `B` cùng sở hữu `std::shared_ptr` trỏ chéo lẫn nhau:

$$\text{A} \xrightarrow{\text{shared_ptr}} \text{B} \quad \text{và} \quad \text{B} \xrightarrow{\text{shared_ptr}} \text{A}$$

Bộ đếm Strong Reference Count của cả hai không bao giờ giảm về 0, dẫn tới toàn bộ mạng lưới đối tượng bị **rò rỉ vĩnh viễn trong RAM**!

Giải pháp: Thay thế một trong hai liên kết bằng **`std::weak_ptr<T>`**. `weak_ptr` chỉ đóng vai trò là người quan sát: Nó không làm tăng Strong Reference Count. Khi muốn sử dụng dữ liệu, nó thăng hạng an toàn thành `shared_ptr` thông qua phương thức `wp.lock()`; nếu đối tượng đã bị giải phóng, `lock()` trả về con trỏ rỗng an toàn.

CHƯƠNG 4

Kế thừa (Inheritance)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa: Tính Kế Thừa (Inheritance)

Kế thừa (Inheritance) là cơ chế cho phép một lớp mới (**Lớp dẫn xuất / Lớp con — Derived Class**) tiếp nhận lại toàn bộ cấu trúc dữ liệu và hành vi của một lớp đã tồn tại (**Lớp cơ sở / Lớp cha — Base Class**), đồng thời mở rộng hoặc tái định nghĩa các hành vi chuyên biệt mà không cần viết lại mã nguồn ban đầu.[a]

[a]Nguồn: OOP5-Inheritance_new.pdf - Trang 3-12: Khái niệm thừa kế và quan hệ kế thừa

4.1 Bản Chất Quan Hệ IS-A và HAS-A

Trong thiết kế phần mềm, việc lựa chọn sai lầm giữa Kế thừa và Hợp thành là nguyên nhân hàng đầu dẫn đến các hệ thống giòn vỡ (*Brittle Architecture*):

- **Mối quan hệ IS-A (Là một):** Thể hiện tính phân loại bản thể luận. Ví dụ: *"Hình tròn LÀ MỘT Hình học"* (`Circle IS-A Shape`). Mối quan hệ này phải được biểu diễn bằng **Kế thừa Công khai (Public Inheritance)**. Đối tượng lớp con phải thay thế được hoàn toàn cho đối tượng lớp cha trong mọi hoàn cảnh mà không phá vỡ tính đúng đắn của chương trình (Nguyên lý Thay thế Liskov - Liskov Substitution Principle).
- **Mối quan hệ HAS-A (Có một / Sở hữu):** Thể hiện sự hợp thành của các thành phần. Ví dụ: *"Ô tô CÓ MỘT Động cơ"* (`Car HAS-A Engine`). Mối quan hệ này bắt buộc phải được mô hình hóa bằng **Hợp Thành (Composition)** hoặc **Tụ Hợp (Aggregation)**: Đưa đối tượng thành viên vào trong lớp, tuyệt đối không dùng kế thừa!

4.1.1 Bố Cục Bộ Nhớ Đối Tượng Kế Thừa Đơn (Single Inheritance Memory Layout)

Trong C++, lớp con kế thừa từ lớp cha được tổ chức theo nguyên lý lồng ghép cấu trúc nhị phân liên tục trong RAM:

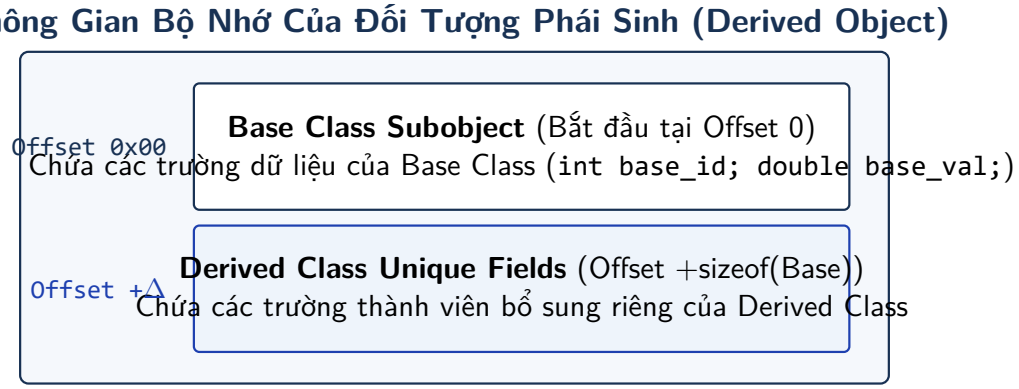


Figure 4.1: Bố cục bộ nhớ nhị phân của đối tượng phái sinh trong Kế thừa Đơn.

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Đặc Điểm Con Trỏ Trong Kế Thừa Đơn

Vì Subobject của Base nằm tại vị trí bắt đầu (Offset 0) của đối tượng Derived, nên:

```
&derived == static_cast<Base*>(&derived)
```

Địa chỉ con trỏ của đối tượng con và địa chỉ con trỏ khi ép kiểu lên đối tượng cha là **hoàn toàn trùng khớp về mặt giá trị số nguyên**! Trình biên dịch không cần sinh thêm bất kỳ lệnh cộng trừ offset nào khi thực hiện phép Upcast trong kế thừa đơn.[a]

[a]Nguồn: OOP5-Inheritance_new.pdf - Trang 13-22: Bố cục bộ nhớ đối tượng kế thừa

4.2 Ma Trận Biến Đổi Quyền Truy Cập Trong Kế Thừa

Quyền truy cập của các thành viên kế thừa từ lớp cha vào lớp con phụ thuộc vào kiểu kế thừa:

Table 4.1: Ma trận biến đổi quyền truy cập các trường dữ liệu trong kế thừa

Quyền Tại Lớp Base	Kế Thừa Public	Kế Thừa Protected	Kế Thừa Private
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	*Bị ẩn hoàn toàn*	*Bị ẩn hoàn toàn*	*Bị ẩn hoàn toàn*

- **Kế thừa Public:** Thiết lập mối quan hệ IS-A thực sự. Cho phép chuyển đổi con trỏ tự do từ con lên cha ở bất kỳ đâu.
- **Kế thừa Private:** Thiết lập mối quan hệ *"Được cài đặt bằng cách sử dụng"* (*Implemented-in-terms-of*). Toàn bộ giao diện của cha bị đóng kín bên trong con.
- **Tối ưu hóa Lớp Cơ sở Rỗng (Empty Base Optimization - EBO):** Theo chuẩn C++, một lớp hoàn toàn rỗng không chứa biến thành viên vẫn phải có kích thước tối thiểu sizeof = 1 byte để đảm bảo mỗi đối tượng có một địa chỉ phân biệt. Tuy nhiên, khi kế thừa theo dạng `private`, trình biên dịch C++ áp dụng EBO để tiêu trừ byte này: Kích thước của lớp con **không bị tăng thêm 1 byte nào**! Kỹ thuật này được STL sử dụng triệt để trong các bộ nén con trỏ và bộ cấp phát tĩnh (`std::allocator`).

4.3 Vòng Đời Khởi Tạo và Hủy Kế Thừa

4.3.1 Trình Tự Thực Thi Khởi Tạo và Giải Phóng Bộ Nhớ

Tiên Đề / Định Nghĩa Pháp Lập: Định Đề Bất Biến Về Trình Tự Vòng Đời Kế Thừa

Khi khởi tạo một đối tượng lớp con:

1. **Khởi tạo Lớp Cơ sở (Base First):** Constructor của lớp cha được thực thi đầu tiên để bảo đảm phần nền tảng được định hình vững chắc.
2. **Khởi tạo Các Thành Viên Lớp Con (Member Fields):** Các biến dữ liệu riêng của lớp con được khởi tạo theo thứ tự khai báo.
3. **Thực thi Thân Constructor Lớp Con (Derived Body):** Phần thân hàm tạo của lớp con hoàn thiện các nghiệp vụ cuối cùng.

Khi đối tượng bị tiêu hủy:

Trình tự hủy diễn ra theo đúng quy tắc LIFO (Last-In, First-Out) ngược lại hoàn toàn:

Destructor Lớp Con chạy trước ⟶ Dọn dẹp thành viên ⟶ Destructor Lớp Cha chạy sau cùng![a]

[a] Nguồn: OOP5-Inheritance_new.pdf - Trang 23-38: Thứ tự thực thi Constructor và Destructor

Hành Vi Không Xác Định (Undefined Behavior): Bẫy Tử Thần: Quên Gọi Base Copy Constructor

Khi lập trình viên tự tay viết Copy Constructor cho lớp con:

```
class Base {
public:
    int baseVal;
    Base(int v) : baseVal(v) {}
    Base(const Base& other) : baseVal(other.baseVal) {
        std::cout << "Base copied\n";
```

```
    }
};

class Derived : public Base {
public:
    int derivedVal;
    Derived(int b, int d) : Base(b), derivedVal(d) {}

    // DANGEROUS ERROR: Forgot to invoke Base(other) in initializer list!
    Derived(const Derived& other) : derivedVal(other.derivedVal) {
        // Compiler will silently invoke Base::Base() default constructor!
        // If Base has no default ctor -> Compilation error.
        // If Base has default ctor -> baseVal is reset, losing copied data!
    }

    // CORRECT CANONICAL IMPLEMENTATION:
    // Derived(const Derived& other) : Base(other), derivedVal(other.
    derivedVal) {}
};
```

4.4 Hiện Tượng Che Khuất Tên (Name Hiding) vs Nạp Chồng

Trong C++, cơ chế phân giải tên (*Name Lookup*) tuân theo nguyên tắc: **Dừng lại ở phạm vi gần nhất tìm thấy định danh trùng khớp!**

```
class Base {
public:
    void process(int x) { std::cout << "Base::process(int)\n"; }
    void process(double x) { std::cout << "Base::process(double)\n"; }
};

class Derived : public Base {
public:
    // Hides all Base::process overloads:
    void process(std::string s) { std::cout << "Derived::process(string)\n"; }
};

void test() {
    Derived d;
    d.process("Hello"); // VALID: Calls Derived::process(string)
    // d.process(10);   // COMPILATION ERROR: Compiler cannot find process(int)!
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Bản Chất Của Hiện Tượng Name Hiding

Tại sao `d.process(10)` không gọi `Base::process(int)`?

1. Khi phân tích cú pháp biểu thức `d.process(10)`, trình biên dịch bắt đầu tìm kiếm tên định danh `process` từ phạm vi của lớp `Derived`.

2. Nó tìm thấy `void Derived::process(std::string)`.
3. **Cơ chế dừng tìm kiếm:** Khi đã tìm thấy ít nhất một tên khớp trong một phạm vi, trình biên dịch **DỪNG TÌM KIẾM NGAY LẬP TỨC** và không bao giờ quét tiếp lên phạm vi của lớp cha `Base`!
4. Sau đó, nó tiến hành phân giải nạp chồng (*Overload Resolution*) trên tập hợp các hàm tìm được tại lớp con. Vì tập hợp này chỉ có phiên bản nhận `std::string`, việc truyền số nguyên `10` bị báo lỗi biên dịch!
5. **Giải pháp chuẩn mực:** Sử dụng từ khóa **`using`** để kéo toàn bộ họ hàm của cha vào lớp con:

```
    class Derived : public Base {
    public:
        using Base::process; // Pull all Base::process overloads into
    Derived scope
        void process(std::string s);
    };

```

Dòng lệnh `using Base::process` tái lập khả năng nạp chồng hoàn hảo giữa cha và con![a]

[a]Nguồn: OOP5-Inheritance_new.pdf - Trang 39-48: Hiện tượng Name Hiding và từ khóa using

4.5 Upcasting, Downcasting và Hiểm Họa Cắt Xén Đối Tượng

4.5.1 Cắt Xén Đối Tượng (Object Slicing)

Hành Vi Không Xác Định (Undefined Behavior): Thảm Họa Cắt Xén Đối Tượng Khi Truyền Tham Trị

Xét đoạn mã sau:

```
void renderEntity(Base b) { // DANGEROUS: Passed by VALUE
    // ...
}

void execution() {
    Derived d(10, 20);
    renderEntity(d); // OBJECT SLICING OCCURS HERE!
}
```

Giải phẫu cơ chế cắt xén dưới RAM:

1. Hàm `renderEntity` yêu cầu một biến có kích thước đúng bằng sizeof(Base) trên ngăn xếp Stack của nó.
2. Khi truyền đối tượng `d` vào, trình biên dịch gọi Copy Constructor của lớp cơ sở: `Base::Base(const Base&)`.
3. Trình biên dịch sao chép chính xác phần Subobject của Base từ `d`, và **GỌT**

BỎ HOÀN TOÀN (SLICE OFF) toàn bộ các trường dữ liệu riêng và bản chất đa hình của `Derived`!

4. Mọi hành vi đa hình và dữ liệu của lớp con bị tiêu biến hoàn toàn mà không có bất kỳ cảnh báo biên dịch nào!
5. **Quy tắc Vàng:** Trong hướng đối tượng, luôn luôn truyền đối tượng đa hình qua **Tham Chiếu Hằng (`const Base&`)** hoặc **Con Trỏ (`Base*`)**, tuyệt đối cấm truyền theo giá trị![a]

[a]Nguồn: OOP5-Inheritance_new.pdf - Trang 49-62: Upcasting, Object Slicing và Downcasting

4.6 Đa Kế Thừa & Bài Toán Kim Cương (Diamond Problem)

4.6.1 Mô Hình Bài Toán Kim Cương

Đa kế thừa (*Multiple Inheritance*) cho phép một lớp kế thừa từ nhiều lớp cha độc lập. Tuy nhiên, nó dẫn tới cấu trúc đồ thị hình thoi kinh điển:

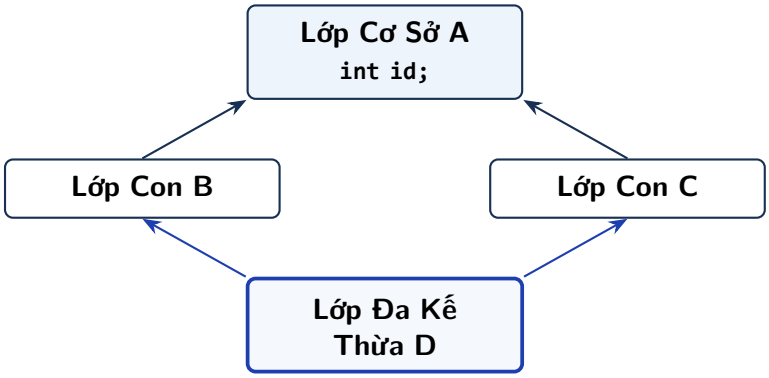


Figure 4.2: Mô hình phân cấp kế thừa hình thoi (The Diamond Problem).

- **Nếu không có kế thừa ảo:** Đối tượng `D` trong RAM sẽ chứa **HAI BẢN SAO ĐỘC LẬP CỦA** A (một bản đi qua nhánh B, một bản đi qua nhánh C). Kích thước đối tượng bị nhân đôi, và lời gọi `d.id` bị trình biên dịch từ chối ngay lập tức vì không thể xác định đang truy cập `id` của nhánh B hay nhánh C (*Ambiguity Error*).
- **Giải pháp: Kế Thừa Ảo (`virtual public A`):**

```
class A { public: int id; A(int x) : id(x) {} };
class B : virtual public A { public: B() : A(1) {} };
class C : virtual public A { public: C() : A(2) {} };
class D : public B, public C {
public:
    // MOST-DERIVED CLASS EXCLUSIVE DUTY:
    // D is directly responsible for initializing A, ignoring calls from B and C!
    D() : A(100), B(), C() {}
};
```

```
10
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Kiến Trúc Bộ Nhớ Kế Thừa Ảo (vbptr & vbtbl)

Để đảm bảo trong lòng đối tượng D chỉ tồn tại duy nhất **MỘT Subobject** A **dùng chung**, trình biên dịch C++ thay đổi hoàn toàn kiến trúc bộ nhớ:

1. Subobject A ảo được đẩy xuống tận cùng của cấu trúc đối tượng D.
2. Các lớp trung gian B và C được chèn thêm một con trỏ cơ sở ảo: **`vbptr` (Virtual Base Pointer)**.
3. `vbptr` trỏ tới một bảng độ lệch **Virtual Base Table (vbtbl)**. Tại runtime, khi B hoặc C muốn truy cập thành viên của A, CPU đọc bảng `vbtbl` để tính toán khoảng dịch chuyển bộ nhớ động tới vị trí thực tế của A trong D.
4. Do đó, kế thừa ảo làm tăng kích thước đối tượng thêm từ 8 đến 16 bytes (cho các con trỏ `vbptr`) và làm chậm tốc độ truy cập dữ liệu do phải qua thêm một bước gián tiếp.[a]

[a] Nguồn: OOP5-Inheritance_new.pdf - Trang 71-76: Đa thừa kế, Sự mơ hồ & Lớp cơ sở ảo

CHƯƠNG 5

Đa hình & Liên kết Động (Polymorphism & Dynamic Binding)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa: Tính Đa Hình (Polymorphism)

Đa hình (Polymorphism) — xuất phát từ tiếng Hy Lạp cổ mang nghĩa "nhiều hình thái" — là trụ cột thứ ba và là linh hồn của Lập trình Hướng Đối tượng. Đa hình cho phép một biến con trỏ hoặc tham chiếu thuộc lớp cơ sở (`Base`) có khả năng trỏ tới đối tượng của bất kỳ lớp phái sinh nào (`Derived`), và khi thực hiện lời gọi hàm thành viên, phiên bản hàm chính xác tương ứng với kiểu đối tượng thực tế sẽ được kích hoạt.[a]

[a]Nguồn: OOP6-Polymorphism_new.pdf - Trang 3-7: Khái niệm tính đa hình trong lập trình hướng đối tượng

5.1 Triết lý Đa hình & Cơ chế Liên kết (Binding)

Để hiểu sâu sắc bản chất của Đa hình, ta phải phân tích sự khác biệt cốt tử giữa hai cơ chế liên kết lời gọi hàm với mã máy:

1. **Liên kết Tĩnh (Early Binding / Static Binding):**
    - Xảy ra tại **thời điểm biên dịch (Compile-time)**. Trình biên dịch dựa hoàn toàn vào **kiểu dữ liệu tĩnh (Static Type)** của biến hoặc con trỏ để xác định địa chỉ hàm cần thực thi.
    - Mã máy sinh ra là một lệnh nhảy trực tiếp: `call 0x004012A0`.
    - *Ưu điểm:* Tốc độ thực thi cực nhanh, không tốn thêm bất kỳ byte bộ nhớ nào, cho phép trình biên dịch thực hiện tối ưu hóa nội dòng (**Function**

Inlining).

- *Nhược điểm:* Thiếu sự linh hoạt, không thể thay đổi hành vi theo ngữ cảnh thời gian chạy.

2. **Liên kết Động (Late Binding / Dynamic Binding):**
   - Xảy ra tại **thời điểm thực thi (Runtime)**. Việc quyết định gọi phiên bản hàm nào được hoãn lại cho tới khi chương trình đang chạy, dựa vào **kiểu dữ liệu thực tế (Dynamic Type)** của đối tượng mà con trỏ đang trỏ tới.
   - Mã máy sinh ra là một lệnh nhảy gián tiếp thông qua con trỏ bộ nhớ: `call qword ptr [rax + offset]`.
   - *Ưu điểm:* Tạo ra kiến trúc phần mềm mở rộng cực kỳ mạnh mẽ (Open-Closed Principle: mở rộng bằng cách thêm lớp mới mà không cần sửa code cũ).
   - *Nhược điểm:* Tốn chi phí gián tiếp (Indirection overhead), tăng kích thước đối tượng và vô hiệu hóa inlining.[1]

Tiên Đề / Định Nghĩa Pháp Lập: Nguyên Lý Zero-Overhead Của C++

Trong các ngôn ngữ như Java hay C#, mọi phương thức thành viên mặc định đều là **Dynamic Binding**. Ngược lại, triết lý thiết kế của C++ kiên định với tôn chỉ **Zero-Overhead Principle**: *"Bạn không phải trả giá cho những gì bạn không sử dụng"*.
Do đó, trong C++, mọi hàm thành viên mặc định **100% là Static Binding**. Chỉ khi lập trình viên chủ động gắn từ khóa `virtual`, trình biên dịch mới kích hoạt cơ chế Dynamic Binding!

5.2 Hàm Ảo, `override` và `final` trong C++11

5.2.1 Cơ chế Hàm Ảo (`virtual`)

Khi khai báo một phương thức với từ khóa `virtual` ở lớp cơ sở, ta đã thiết lập một "hợp đồng đa hình":

```
class Shape {
public:
    virtual void draw() const {
        std::cout << "Drawing generic shape\n";
    }
    virtual ~Shape() = default;
};

class Circle : public Shape {
public:
    void draw() const override { // Override virtual method
        std::cout << "Drawing Circle\n";
```

[1] Nguồn: OOP6-Polymorphism_new.pdf - Trang 8-15: Early Binding vs Late Binding

```
    }
};
```

Quy tắc lan truyền ảo (Virtual Propagation): Một khi một phương thức đã được khai báo là `virtual` ở lớp cơ sở, nó sẽ **tự động mang tính ảo vĩnh viễn** trong toàn bộ cây thừa kế của các lớp con cháu, cho dù ở lớp con ta có viết lại từ khóa `virtual` hay không.[2]

5.2.2 Bộ Đôi Từ Khóa Ngữ Cảnh C++11: `override` và `final`

Trước C++11, lỗi ngầm phổ biến nhất của lập trình viên là viết sai chữ ký hàm con dẫn đến việc vô tình tạo ra một hàm mới thay vì ghi đè hàm cha:

```
class Base {
public:
    virtual void process(int x) const;
};

// C++98: Signature mismatch pitfall
class Derived : public Base {
public:
    void process(long x) const; // LOGICAL FLAW: Overloads instead of overriding!
    void process(int x);        // LOGICAL FLAW: Missing const qualifier!
};
```

Để chấm dứt thảm họa này, chuẩn C++11 bổ sung hai từ khóa ngữ cảnh (*contextual keywords*):

- **`override`**: Yêu cầu trình biên dịch kiểm tra nghiêm ngặt xem hàm này có thực sự ghi đè một hàm ảo nào từ lớp cha hay không. Nếu chữ ký hàm, kiểu trả về, hoặc thuộc tính `const` không khớp 100%, trình biên dịch lập tức báo lỗi biên dịch (*Compilation Error*):

```
    class DerivedC11 : public Base {
    public:
        void process(int x) const override; // VALID: Perfectly matches Base signature
        // void process(long x) const override; // COMPILATION ERROR: Detected immediately!
    };

```

- **`final`**: Mang hai mục đích kiểm soát kiến trúc:
  1. Khóa không cho lớp con tiếp tục ghi đè phương thức ảo:

```
        virtual void process(int x) const override final; // Blocks further overriding

```

  2. Khóa không cho phép bất kỳ lớp nào kế thừa từ lớp này nữa:

[2]Nguồn: OOP6-Polymorphism_new.pdf - Trang 16-22: Khai báo và ghi đè hàm ảo

```
        class SecureKey final : public Base { /* ... */ }; // Prohibits inheritance

```

Tối ưu hóa Devirtualization: Khi trình biên dịch thấy một hàm hoặc lớp được đánh dấu `final`, nó có thể biến đổi các lời gọi gián tiếp thành lệnh gọi trực tiếp (**Direct Call**), loại bỏ hoàn toàn chi phí tra cứu bảng ảo![3]

5.3 Phân tích Dưới Gầm Máy: Bảng Ảo (VTable) & Con trỏ vptr

Làm thế nào mà CPU có thể tìm ra đúng hàm `Circle::draw()` khi nó chỉ đang giữ một con trỏ kiểu `Shape*`? Đây là bí ẩn cấu trúc sâu sắc nhất của C++.

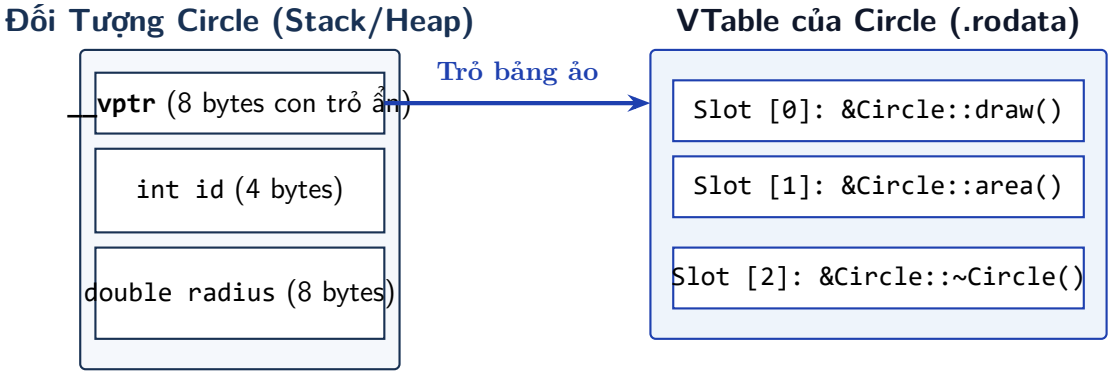


Figure 5.1: Cấu trúc nhị phân của Bảng Phương Thức Ảo (VTable) và con trỏ `__vptr` trong bộ nhớ.

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Kiến Trúc Bảng Ảo VTable và Con Trỏ vptr Dưới RAM

Để hiện thực hóa Dynamic Binding mà không phải trả giá bằng việc kiểm tra kiểu dạng chuỗi chậm chạp (`instanceof`), trình biên dịch C++ sử dụng một cấu trúc dữ liệu nhị phân nội tại:

1. **Virtual Method Table (VTable):**
   - Là một mảng tĩnh các con trỏ hàm (`const void*[]`), được trình biên dịch sinh ra duy nhất một bản cho **mỗi lớp có ít nhất một hàm ảo**.
   - VTable được lưu trữ trong phân vùng bộ nhớ chỉ đọc (**Read-Only Data Segment** — `.rodata` trên Linux ELF hoặc `.rdata` trên Windows PE). Nó tồn tại suốt vòng đời của tiến trình.
   - Mỗi phần tử trong VTable chứa địa chỉ bắt đầu của mã máy của một phương thức ảo tương ứng theo thứ tự khai báo.
2. **Virtual Table Pointer (vptr):**

[3]Nguồn: OOP6-Polymorphism_new.pdf - Trang 23-28: Từ khóa override và final trong C++11

- Khi một lớp có hàm ảo, trình biên dịch âm thầm chèn thêm một trường dữ liệu ẩn vào đầu đối tượng: con trỏ `__vptr` (kích thước 8 bytes trên nền tảng 64-bit).
- Con trỏ `vptr` này trỏ thẳng tới bảng VTable tương ứng của lớp mà đối tượng thuộc về.
- `vptr` được tự động thiết lập bởi mã máy của **Constructor** ngay khi đối tượng được sinh ra trong RAM.

Giải phẫu Lệnh Nhảy Gián Tiếp (Indirect Dispatching): Khi thực thi dòng lệnh `pShape->draw()`:

1. CPU lấy địa chỉ của đối tượng từ thanh ghi con trỏ `pShape` (ví dụ `RCX` hoặc `RDI`).
2. CPU giải tham chiếu offset 0 để lấy giá trị con trỏ `__vptr`: `RAX = *(pShape)`.
3. CPU tra cứu mảng VTable tại vị trí của hàm `draw()` (ví dụ slot 0): `RDX = *(RAX + 0)`.
4. CPU thực hiện lệnh nhảy gián tiếp: `call RDX`, đồng thời truyền con trỏ đối tượng `pShape` làm con trỏ `this` ẩn.[a]

[a] Nguồn: OOP6-Polymorphism_new.pdf - Trang 29-45: Kiến trúc VTable, vptr và Assembly Dispatch

5.3.1 Cơ Chế Điều Chỉnh Con Trỏ Thunk (Adjustor Thunk)

Trong Đa kế thừa (Multiple Inheritance), khi một lớp `C` kế thừa từ cả `A` và `B`:

```
class A { public: virtual void fa() {} };
class B { public: virtual void fb() {} };
class C : public A, public B {
public:
    void fa() override {}
    void fb() override {}
};
```

Trong lòng đối tượng `C`, Subobject `A` nằm ở offset 0, nhưng Subobject `B` nằm ở offset lệch đi một khoảng Δ (ví dụ +8 bytes). Khi ta gọi:

```
B* pb = new C();
pb->fb(); // Invokes virtual method of B on object C
```

Lúc này, thanh ghi con trỏ truyền vào hàm `C::fb()` đang trỏ vào Subobject `B` (tại offset +8), trong khi mã máy của `C::fb()` lại kỳ vọng con trỏ `this` trỏ vào đầu đối tượng toàn vẹn `C` (tại offset 0)!

Giải pháp Thunk: Trình biên dịch sinh ra một đoạn mã đệm siêu nhỏ gọi là **Adjustor Thunk**. Trong VTable của Subobject `B`, địa chỉ hàm không trỏ thẳng vào `C::fb()`, mà trỏ vào Thunk:

```
__thunk_for_C_fb:
    sub rcx, 8          ; Rewind this pointer to offset 0 of full object C
    jmp C::fb           ; Direct jump to actual implementation
```

Đây là minh chứng tuyệt đỉnh cho sự tinh vi của các kỹ sư thiết kế trình biên dịch C++ để bảo đảm an toàn bộ nhớ tuyệt đối.

5.4 Hàm Thuần Ảo & Lớp Cơ Sở Trừu Tượng (Abstract Base Classes)

Tiên Đề / Định Nghĩa Pháp Lập: Lớp Trừu Tượng (Abstract Base Class - ABC)

Một hàm ảo được gán cú pháp `= 0` ở cuối khai báo được gọi là một **Hàm thuần ảo (Pure Virtual Function)**.
Một lớp chứa ít nhất một hàm thuần ảo được định nghĩa là một **Lớp Cơ sở Trừu tượng (Abstract Base Class)**. Đặc tính tối thượng của lớp trừu tượng là:

- **Tuyệt đối không thể khởi tạo thực thể (Cannot instantiate)** trực tiếp trên Stack hoặc Heap. Cố tình viết `Shape s;` sẽ dẫn đến lỗi biên dịch ngay lập tức.
- Lớp con kế thừa từ ABC bắt buộc phải ghi đè (`override`) và hiện thực hóa toàn bộ các hàm thuần ảo; nếu bỏ sót dù chỉ một hàm, lớp con đó tiếp tục bị coi là lớp trừu tượng![a]

[a] Nguồn: OOP6-Polymorphism_new.pdf - Trang 46-55: Hàm thuần ảo và lớp cơ sở trừu tượng

```cpp
class IDevice {
public:
    virtual void boot() = 0;      // Pure virtual function
    virtual void shutdown() = 0;  // Pure virtual function
    virtual ~IDevice() = default;
};
```

5.4.1 Bẫy Tinh Vi: Hàm Thuần Ảo Vẫn Có Thể Có Body!

Rất nhiều lập trình viên C++ nhầm tưởng rằng hàm thuần ảo (`= 0`) thì không được phép có phần thân code (body). Đây là một sai lầm phổ biến!

Chuẩn C++ cho phép hoàn toàn việc viết định nghĩa (body) cho một hàm thuần ảo, miễn là phần định nghĩa đó nằm **bên ngoài khai báo của lớp**:

```cpp
class BaseAbstract {
public:
    virtual void logStatus() = 0; // Pure virtual, BaseAbstract is ABC!
    virtual ~BaseAbstract() = default;
};

// Define body for pure virtual function OUTSIDE class declaration:
void BaseAbstract::logStatus() {
    std::cout << "[BASE AUDIT LOG] Standard audit log record\n";
}

```

```cpp
class ConcreteChild : public BaseAbstract {
public:
    void logStatus() override {
        // Reuse base default implementation, then add custom logic:
        BaseAbstract::logStatus();
        std::cout << "[CHILD LOG] Custom specialized processing\n";
    }
};
```

Ý nghĩa kiến trúc: Kỹ thuật này cho phép kiến trúc sư hệ thống vừa **ép buộc lớp con phải chủ động override** (không thể vô tình quên), vừa **cung cấp sẵn mã nguồn mặc định tái sử dụng** cho các lớp con gọi khi cần thiết.

5.4.2 Pure Virtual Destructor: Phải Luôn Có Body!

Nếu bạn muốn tạo một Abstract Base Class nhưng lớp của bạn không có hàm nghiệp vụ nào để gán `= 0`, bạn có thể biến Destructor thành thuần ảo:

```cpp
class InterfaceOnly {
public:
    virtual ~InterfaceOnly() = 0; // Pure virtual destructor makes class ABC
};

// MANDATORY DEFINITION OUTSIDE CLASS BODY:
InterfaceOnly::~InterfaceOnly() {}
```

Cảnh Báo Kỹ Thuật & Rủi Ro Biên Dịch:

Nếu bạn quên không viết định nghĩa rỗng `InterfaceOnly::~InterfaceOnly() {}`, trình biên dịch sẽ cho qua nhưng **Trình liên kết (Linker) sẽ báo lỗi sập hệ thống (Undefined Reference / LNK2019)** ngay khi bất kỳ lớp con nào bị hủy, bởi vì trong chuỗi hủy kế thừa, Destructor của lớp con luôn tự động gọi ngầm Destructor của lớp cha!

5.5 Destructor Ảo (Virtual Destructor) — Luật Sống Còn

Đây là câu hỏi phỏng vấn kinh điển và là bẫy rò rỉ bộ nhớ phổ biến nhất trong các dự án C++ thực tế. Hãy quan sát đoạn mã thảm họa sau:

```cpp
class BaseResource {
public:
    BaseResource() { std::cout << "Base acquired\n"; }
    ~BaseResource() { std::cout << "Base freed\n"; } // Non-virtual destructor!
};

class DerivedResource : public BaseResource {
private:
    int* largeBuffer;
public:
    DerivedResource() : largeBuffer(new int[1000000]) {
```

```
        std::cout << "Derived allocated 4MB RAM\n";
    }
    ~DerivedResource() {
        delete[] largeBuffer; // Free 4MB RAM
        std::cout << "Derived buffer freed successfully\n";
    }
};

void cleanup() {
    BaseResource* ptr = new DerivedResource();
    delete ptr; // THAM HOA RO RI BO NHO XAY RA TAI DAY!
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Phân Tích Cơ Chế Thảm Họa Khi Thiếu Virtual Destructor

Tại dòng lệnh `delete ptr;`:

1. Con trỏ `ptr` có kiểu tĩnh là `BaseResource*`.
2. Vì Destructor `~BaseResource()` **KHÔNG CÓ TỪ KHÓA `virtual`**, trình biên dịch áp dụng cơ chế **Static Binding (Early Binding)**.
3. Trình biên dịch sinh mã máy gọi trực tiếp địa chỉ của hàm `BaseResource::~BaseResource()`.
4. **Hậu quả:** Hàm `DerivedResource::~DerivedResource()` **HOÀN TOÀN BỊ BỎ QUA**! Lệnh `delete[] largeBuffer` không bao giờ được chạy. Toàn bộ 4 Megabytes RAM bị rò rỉ vĩnh viễn trong không gian nhớ của hệ điều hành!
5. Tệ hơn nữa, chuẩn ISO C++11 quy định: Việc xóa một đối tượng phái sinh thông qua con trỏ lớp cơ sở không có virtual destructor dẫn đến **Hành Vi Không Xác Định (Undefined Behavior)**.[a]

[a] Nguồn: OOP6-Polymorphism_new.pdf - Trang 59-65: Virtual Destructor và Hiểm họa rò rỉ bộ nhớ

Tiên Đề / Định Nghĩa Pháp Lập: Quy Tắc Vàng Của Scott Meyers Về Destructor

- **Quy tắc 1:** Bất kỳ lớp nào được thiết kế để làm lớp cơ sở có tính đa hình (*Polymorphic Base Class*), Destructor của nó **BẮT BUỘC PHẢI LÀ `public virtual`**.
- **Quy tắc 2:** Nếu một lớp không được thiết kế để làm lớp cha đa hình (hoặc không muốn cho phép người dùng xóa qua con trỏ lớp cơ sở), hãy khai báo Destructor là **`protected non-virtual`**. Cách này ngăn chặn hoàn toàn việc gọi `delete ptrBase` từ bên ngoài, triệt tiêu lỗi rò rỉ ngay tại compile-time!

5.6 Bẫy Tử Thần: Gọi Hàm Ảo Trong Constructor & Destructor

Trong các ngôn ngữ như Java hoặc C#, gọi một phương thức ảo trong Constructor sẽ nhảy ngay tới phiên bản của lớp con. Tuy nhiên, trong C++, hành động này là một **Bẫy chết người** có thể làm sập toàn bộ ứng dụng!

```cpp
class Transaction {
public:
    Transaction() {
        logCreation(); // Pitfall: Virtual call in Constructor!
    }
    virtual void logCreation() const {
        std::cout << "Base Transaction logged\n";
    }
    virtual ~Transaction() = default;
};

class BuyTransaction : public Transaction {
private:
    std::string stockCode;
public:
    BuyTransaction(const std::string& code) : stockCode(code) {}
    void logCreation() const override {
        std::cout << "BuyTransaction logged for: " << stockCode << "\n";
    }
};
```

Khi ta thực thi lệnh: `BuyTransaction tx("AAPL");`, điều gì sẽ in ra màn hình?

- Đa số lập trình viên mới nghĩ rằng nó sẽ in: `"BuyTransaction logged for: AAPL"`.
- **KẾT QUẢ THỰC TẾ:** Màn hình in ra `"Base Transaction logged"`! Đa hình đã hoàn toàn biến mất!

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Nguyên Nhân Gầm Máy: Sự Tiến Hóa Của vptr Trong Vòng Đời Đối Tượng

Hãy nhìn vào tiến trình khởi tạo bộ nhớ từng bước của CPU:

1. Để khởi tạo `BuyTransaction`, trước hết Constructor của `Transaction` (lớp cha) phải được thực thi.
2. Tại thời điểm Constructor của `Transaction` đang chạy, **phần dữ liệu của `BuyTransaction` (chuỗi `stockCode`) CHƯA ĐƯỢC CẤP PHÁT VÀ CHƯA ĐƯỢC KHỞI TẠO**.
3. Do đó, con trỏ `__vptr` của đối tượng lúc này **đang trỏ tới VTable của `Transaction`**, chứ CHƯA trỏ tới VTable của `BuyTransaction`!
4. Trình biên dịch C++ cố tình cô lập lời gọi này về dạng **Static Binding** của lớp cha để bảo vệ bạn: Nếu nó cho phép gọi `BuyTransaction::logCreation()`, hàm này sẽ truy cập vào biến `stockCode` chưa tồn tại, dẫn đến đọc rác bộ nhớ hoặc làm sập chương

trình (Segmentation Fault)!

Thảm Họa Crash Runtime Với Hàm Thuần Ảo (`__cxa_pure_virtual`): Nếu phương thức ảo được gọi trong Constructor là **Hàm thuần ảo (`= 0`)**:

```
class Engine {
public:
    Engine() { start(); } // start() is pure virtual!
    virtual void start() = 0;
};
```

VTable của `Engine` tại slot của `start` chứa con trỏ trỏ tới hàm xử lý lỗi nội bộ của trình biên dịch: `__cxa_pure_virtual`. Khi chương trình chạy qua dòng này, hệ điều hành lập tức chấm dứt tiến trình và ném lỗi kinh hoàng:[a]

`pure virtual method called - terminate called without an active exception (Aborted)`

Điều tương tự diễn ra chính xác trong **Destructor**: Khi Destructor của lớp con chạy xong, đối tượng bị "hạ cấp" về lớp cha và `vptr` bị đổi ngược về VTable của lớp cha trước khi chạy Destructor của cha!

[a] Nguồn: OOP6-Polymorphism_new.pdf - Trang 66-72: Pitfall - Virtual Function in Constructor/Destructor

CHƯƠNG 6

Khuôn mẫu & Lập trình Tổng quát (Templates & Generic Programming)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa: Lập Trình Tổng Quát (Generic Programming)

Lập trình Tổng quát (Generic Programming) là hệ hình lập trình trong đó mã nguồn được viết một cách trừu tượng hóa, độc lập hoàn toàn với các kiểu dữ liệu cụ thể. Thay vì phải sao chép thuật toán tìm kiếm hay cấu trúc danh sách liên kết cho từng kiểu `int`, `double`, `Student`, lập trình viên sử dụng **Khuôn mẫu (Templates)** để biến **Kiểu dữ liệu thành Tham số (Type-as-a-Parameter)**.[a]

[a]Nguồn: OOP7-Template.pdf - Trang 2-9: Khái niệm khuôn mẫu hàm và khuôn mẫu lớp

6.1 Khuôn Mẫu Hàm & Khuôn Mẫu Lớp (Function & Class Templates)

6.1.1 Cú pháp và Sự tương đương giữa `typename` và `class`

```
// Generic swap function template
template <typename T>
void genericSwap(T& a, T& b) {
    T temp = std::move(a);
    a = std::move(b);
    b = std::move(temp);
}

// Generic safe stack class template
template <typename T>
class SafeStack {
```

```
private:
    std::vector<T> elements;
public:
    void push(const T& val) { elements.push_back(val); }
    void push(T&& val) { elements.push_back(std::move(val)); }
    T pop() {
        if (elements.empty()) throw std::out_of_range("Stack empty");
        T val = std::move(elements.back());
        elements.pop_back();
        return val;
    }
};
```

Phân biệt `typename` *và* `class` *trong khai báo template:* Trong danh sách tham số kiểu khuôn mẫu `template <typename T>` và `template <class T>`, hai từ khóa này **hoàn toàn đồng nghĩa 100%** về mặt ngữ nghĩa ngữ pháp. Từ khóa `typename` được chuẩn hóa sau này để tránh gây hiểu lầm rằng kiểu truyền vào bắt buộc phải là một `class` (vì các kiểu nguyên thủy như `int`, `double`, con trỏ thô đều truyền vào được).

6.2 Quá Trình Biên Dịch Hai Pha (Two-Phase Lookup) & Dependent Types

Một trong những cơ chế phức tạp nhất của trình biên dịch C++ khi xử lý template là quy trình phân giải tên hai pha:

1. **Pha 1 (Pha định nghĩa khuôn mẫu - Parsing Phase):**
    - Xảy ra khi trình biên dịch lần đầu tiên quét qua đoạn mã `template <typename T> ...`.
    - Trình biên dịch chỉ kiểm tra cú pháp cơ bản độc lập với kiểu (dấu chấm phẩy, đóng mở ngoặc, các biến không phụ thuộc vào `T`).
2. **Pha 2 (Pha tạo thực thể - Instantiation Phase):**
    - Xảy ra khi lập trình viên thực sự sử dụng template với một kiểu cụ thể (ví dụ `SafeStack<int> s;`).
    - Trình biên dịch thay thế toàn bộ tham số `T` bằng `int` và tiến hành biên dịch mã máy thực tế, kiểm tra tính hợp lệ của các phép toán trên kiểu đó.[1]

6.2.1 Bắt Buộc Sử Dụng Từ Khóa `typename` Cho Kiểu Phụ Thuộc (Dependent Types)

Xét đoạn mã template kinh điển sau:

```
template <typename Container>
void printFirstElement(const Container& c) {
    // COMPILATION ERROR IF MISSING 'typename':
    typename Container::const_iterator it = c.begin();
    if (it != c.end()) {
```

[1]Nguồn: OOP7-Template.pdf - Trang 15-24: Quá trình biên dịch Template và Two-Phase Lookup

```
        std::cout << *it << "\n";
    }
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Tại Sao Bắt Buộc Viết `typename Container::const_iterator`?

Trong Pha 1 của quá trình biên dịch, trình biên dịch chưa hề biết kiểu cụ thể của `Container` là gì.
Xét biểu thức có dạng: `Container::foo * bar;`

- **Trường hợp A:** Nếu `foo` là một **Kiểu dữ liệu** (Type), đây là dòng lệnh khai báo một con trỏ `bar` có kiểu là `Container::foo`.
- **Trường hợp B:** Nếu `foo` là một **Biến tĩnh** (Static Data Member), đây là một **Phép nhân số học**: Lấy giá trị biến tĩnh `foo` nhân với biến `bar`!

Theo quy tắc chuẩn của ISO C++, khi đứng trước một tên phụ thuộc (*Dependent Name*), trình biên dịch **mặc định coi đó là một biến tĩnh**.
Do đó, nếu `Container::const_iterator` thực sự là một kiểu dữ liệu nội bộ (Nested Type), bạn **bắt buộc phải gắn từ khóa `typename` ở phía trước** để thông báo cho trình biên dịch: *"Đây là một kiểu dữ liệu, hãy phân tích cú pháp theo ngữ cảnh khai báo kiểu!"*[a]

[a]Nguồn: OOP7-Template.pdf - Trang 25-28: Từ khóa typename cho dependent types

6.3 Cơ Chế Tạo Thực Thể & Hiện Tượng Phình To Mã (Code Bloat)

6.3.1 Tại Sao Phải Viết Toàn Bộ Code Template Trong File Header (`.hpp`)?

Đây là lỗi phổ biến nhất của sinh viên chuyển từ lập trình hướng thủ tục sang lập trình hướng đối tượng với template: Tách khai báo vào file `.h` và viết định nghĩa thân hàm vào file `.cpp`.

```
// File: MyStack.h
template <typename T>
class MyStack {
public:
    void push(T val);
};

// File: MyStack.cpp
#include "MyStack.h"
template <typename T>
void MyStack<T>::push(T val) { /* logic */ } // ERROR ARCHITECTURE!

// File: main.cpp
#include "MyStack.h"
int main() {
    MyStack<int> s;
    s.push(10); // LINKER ERROR: undefined reference to MyStack<int>::push(int)
```

```
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Nguyên Nhân Gầm Máy: Mô Hình Biên Dịch Độc Lập (Separate Compilation)

1. C++ biên dịch từng file mã nguồn (`.cpp`) thành các đơn vị dịch độc lập (**Translation Units - TU**) để tạo ra các file đối tượng (`.obj` / `.o`).
2. Khi biên dịch file `MyStack.cpp`, trình biên dịch thấy template `MyStack<T>`, nhưng vì **không có mã nào gọi `MyStack<int>`** trong file này, trình biên dịch **không thể sinh ra một dòng mã máy nào**! File `MyStack.o` hoàn toàn trống rỗng!
3. Khi biên dịch file `main.cpp`, trình biên dịch thấy lệnh gọi `s.push(10)`, nhưng vì file `MyStack.h` chỉ chứa khai báo mà không có phần thân hàm, nó không thể sinh mã máy được. Nó tạm thời xuất ra một lời gọi hàm ngoại vi chưa phân giải.
4. Đến giai đoạn liên kết (**Linking**), Linker đi tìm mã máy của `MyStack<int>::push(int)` trong toàn bộ các file `.o` nhưng không tìm thấy ở đâu → Báo lỗi chết người: **LNK2019 / Undefined Reference**!
5. **Kết luận:** Template không phải là mã máy, template chỉ là **Bản Thiết Kế (Blueprint)**. Do đó, toàn bộ khai báo và định nghĩa của template **bắt buộc phải nằm trọn vẹn trong file Header (`.hpp`)** để bất kỳ file `.cpp` nào gọi nó cũng có đầy đủ mã nguồn để tự sinh mã máy.[a]

[a] Nguồn: OOP7-Template.pdf - Trang 29-38: Template Compilation Model & Header-only requirement

6.3.2 Hiện Tượng Phình To Mã Nguồn (Code Bloat)

Khi chương trình gọi `SafeStack<int>`, `SafeStack<double>`, `SafeStack<std::string>`, và `SafeStack<void*>`, trình biên dịch sẽ sinh ra **4 bản sao mã máy nhị phân hoàn toàn riêng biệt**. Nếu template của bạn lớn, dung lượng file thực thi (`.exe`) sẽ tăng vọt theo cấp số nhân.

Giải pháp Thin Template Idiom của STL: Đối với các kiểu con trỏ (`T*`), STL sử dụng một lớp cơ sở không phải template thao tác hoàn toàn trên `void*`. Sau đó, các template con trỏ chỉ đơn thuần kế thừa và ép kiểu an toàn:

```
class RawPointerStack { // Compiled ONCE into machine code
protected:
    std::vector<void*> rawPtrs;
    void pushRaw(void* ptr);
    void* popRaw();
};

template <typename T>
class PointerStack : private RawPointerStack { // Inlined, zero overhead
public:
    void push(T* ptr) { pushRaw(static_cast<void*>(ptr)); }
    T* pop() { return static_cast<T*>(popRaw()); }
};
```

6.4 Tham Số Không Phải Kiểu (NTTP) & Template Template Parameters

6.4.1 Tham Số Không Phải Kiểu (Non-type Template Parameters - NTTP)

Template không chỉ nhận kiểu dữ liệu mà còn có thể nhận các giá trị hằng số nguyên, con trỏ, hoặc tham chiếu tại thời điểm biên dịch:

```cpp
// Memory optimization: Zero heap allocation
template <typename T, std::size_t Capacity>
class FixedCapacityArray {
private:
    T data[Capacity]; // Array allocated directly on STACK!
    std::size_t size = 0;
public:
    constexpr std::size_t max_size() const { return Capacity; }
    void push(const T& val) {
        if (size >= Capacity) throw std::overflow_error("Buffer full");
        data[size++] = val;
    }
};

// Usage:
FixedCapacityArray<int, 256> fastBuffer; // 1024 bytes on Stack, 0 ns alloc
    overhead!
```

Đây chính là nền tảng hoạt động của `std::array<T, N>` trong thư viện chuẩn C++11.[2]

6.5 Chuyên Biệt Hóa Khuôn Mẫu (Template Specialization) & Bẫy Herb Sutter

Đôi khi, một thuật toán tổng quát không tối ưu cho một kiểu dữ liệu đặc thù. C++ cho phép ta cung cấp các phiên bản chuyên biệt hóa:

6.5.1 Chuyên Biệt Hóa Toàn Phần & Một Phần (Full vs Partial Specialization)

```cpp
// 1. Primary Template
template <typename T, typename Allocator>
class DynamicList { /* Generic implementation */ };

// 2. Partial Specialization for pointer types:
template <typename T, typename Allocator>
class DynamicList<T*, Allocator> {
    /* Specialized implementation for pointers */
};

// 3. Full / Explicit Specialization for bool:
template <>
```

[2]Nguồn: OOP7-Template.pdf - Trang 39-48: Non-type Template Parameters và ứng dụng

```cpp
class DynamicList<bool, std::allocator<bool>> {
    /* Bit-packing optimization: 1 BIT per boolean instead of 1 BYTE! */
};
```

6.5.2 Bẫy Tử Thần Herb Sutter: Function Template Không Hỗ Trợ Partial Specialization!

Chuẩn C++ **TUYỆT ĐỐI CẤM** việc Chuyên biệt hóa một phần đối với Khuôn mẫu hàm (*Function Template Partial Specialization is illegal*).

Nhiều lập trình viên cố gắng kết hợp Nạp chồng hàm template (Function Template Overloading) với Chuyên biệt hóa toàn phần (Full Specialization), tạo ra các lỗi logic thảm họa:

```cpp
template <typename T> void process(T);       // (1) Primary template 1
template <typename T> void process(T*);      // (2) Primary template 2 (Overload)

template <> void process<int*>(int*);        // (3) Specialization of which?
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Bẫy Sutter's Trap Dưới Lăng Kính Phân Giải Overload

Theo chuẩn ISO C++: **Quá trình phân giải nạp chồng (Overload Resolution) chỉ xem xét các Primary Templates!**

1. Trình biên dịch xem xét (1) và (2) để chọn ra template phù hợp nhất.
2. Nếu hàm (3) được khai báo sau (2), nó là specialization của (2). Nhưng nếu nó khai báo trước (2), nó lại trở thành specialization của (1)!
3. Khi có sự xuất hiện của các chuyển đổi kiểu ngầm định, hàm specialization có thể bị bỏ qua hoàn toàn, dẫn đến trình biên dịch gọi một template mà lập trình viên không hề mong muốn.
4. **Quy Tắc Herb Sutter (C++ Guru):** *"Không bao giờ chuyên biệt hóa khuôn mẫu hàm. Khi cần đa dạng hóa hành vi của hàm template, hãy nạp chồng hàm bình thường, hoặc ủy quyền (delegate) việc thực thi cho một Class Template nội bộ!"*[a]

[a]Nguồn: OOP7-Template.pdf - Trang 49-62: Chuyên biệt hóa khuôn mẫu và Bẫy nạp chồng Herb Sutter

6.6 Các Tính Năng Khuôn Mẫu C++11 Hiện Đại

6.6.1 Variadic Templates (Khuôn Mẫu Tham Số Biến Thiên)

Trước C++11, để viết một hàm in ra số lượng tham số tùy ý, lập trình viên phải viết hàng chục bản nạp chồng quá tải. C++11 giới thiệu **Variadic Templates** thông qua **Parameter Pack**:

```cpp
// 1. Base case to terminate compile-time recursion
void printAll() {
    std::cout << "\n";
```

```cpp
}

// 2. Recursive pack expansion at COMPILE-TIME
template <typename Head, typename... Tail>
void printAll(const Head& head, const Tail&... tail) {
    std::cout << head << " ";
    printAll(tail...); // Unpack parameter pack recursively
}

// Usage:
printAll(1, 3.14, "C++ Bible", 'A', true);
```

Cơ chế này hoàn toàn không dùng con trỏ ngăn xếp biến thiên kiểu C (`va_list`), đảm bảo **Type-safety 100%** và được tối ưu hóa inline hoàn toàn tại compile-time! Đây là nền tảng xây dựng nên `std::tuple` và phương thức `emplace_back`.[3]

6.6.2 CRTP: Đa Hình Tĩnh (Static Polymorphism)

Làm thế nào để đạt được sức mạnh của Đa hình nhưng **không phải trả giá cho VTable, con trỏ vptr, và indirect jump**? Câu trả lời là kỹ thuật **Curiously Recurring Template Pattern (CRTP)**:

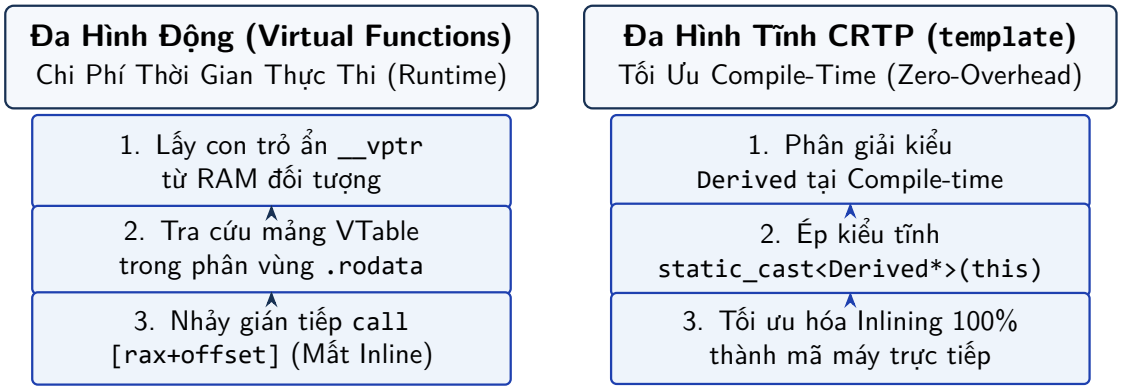


Figure 6.1: So sánh cơ chế phân giải giữa Đa hình động (VTable) và Đa hình tĩnh CRTP.

```cpp
template <typename Derived>
class BaseProcessor {
public:
    void execute() {
        // Static downcast of this pointer to derived class at compile-time:
        static_cast<Derived*>(this)->processImpl();
    }
};

class FastNetworkProcessor : public BaseProcessor<FastNetworkProcessor> {
public:
    void processImpl() { // No virtual keyword needed!
        std::cout << "High-speed zero-overhead processing\n";
    }
};

```

[3]Nguồn: OOP7-Template.pdf - Trang 63-70: C++11 Variadic Templates và Parameter Packs

```
template <typename T>
void run(BaseProcessor<T>& proc) {
    proc.execute(); // Fully inlined, maximum CPU clock execution speed!
}
```

CRTP là vũ khí tối thượng được sử dụng trong các hệ thống giao dịch tài chính tần suất cao (High-Frequency Trading - HFT) và game engine, nơi từng phần tỉ giây (nanosecond) đều quý giá!

CHƯƠNG 7

Xử lý Ngoại lệ & Thư viện Chuẩn STL (Exceptions & Modern STL)

Tiên Đề / Định Nghĩa Pháp Lập: Triết Lý Xử Lý Lỗi Hiện Đại

Trong kỹ thuật lập trình hướng thủ tục cổ điển (như ngôn ngữ C), việc quản lý lỗi dựa hoàn toàn vào **Mã lỗi trả về (Return Error Codes)** và biến toàn cục `errno`. Cách tiếp cận này bộc lộ ba khiếm khuyết mang tính bản chất:

1. **Nguy cơ bỏ quên lỗi:** Lập trình viên dễ dàng bỏ qua việc kiểm tra giá trị trả về của hàm, khiến trạng thái lỗi lan truyền âm thầm và phá hủy dữ liệu hệ thống ở những bước tiếp theo.
2. **Ô nhiễm chữ ký hàm:** Mã lỗi làm trộn lẫn giá trị trả về hợp lệ với trạng thái thất thường, làm phức tạp hóa cấu trúc logic của chương trình (*Happy Path* bị chìm ngập trong các khối `if (err != 0)`).
3. **Sự bất lực trong Constructor: Constructor hoàn toàn không có kiểu trả về**! Do đó, mã lỗi hoàn toàn bất lực trong việc thông báo thất bại khi khởi tạo một đối tượng.[a]

Cơ chế Ngoại lệ (Exception Handling) tách bạch triệt để luồng thực thi nghiệp vụ bình thường ra khỏi luồng cứu vãn sự cố (*Error Recovery Path*), tạo tiền đề cho các hệ thống phần mềm có độ tin cậy cấp công nghiệp.

[a]Nguồn: OOP8-Exception.pdf - Trang 2-12: Triết lý xử lý ngoại lệ và phân loại lỗi

7.1 Cơ Chế Bắt Ngoại Lệ & Bẫy Object Slicing

Khi một đoạn mã phát hiện một tình huống bất thường vượt quá thẩm quyền xử lý nội bộ, nó kích hoạt ngoại lệ bằng lệnh `throw`:

```
#include <iostream>
#include <stdexcept>
#include <string>

```

```cpp
void executeTransaction(int balance, int withdrawalAmount) {
    if (withdrawalAmount > balance) {
        throw std::runtime_error("Insufficient funds: Overdraft limit exceeded!");
    }
    std::cout << "[Banking] Withdrawal successful. Remaining: " << (balance - withdrawalAmount) << "\n";
}

void processAccount() {
    try {
        executeTransaction(500, 1000);
    }
    catch (const std::exception& e) { // STANDARD: Catch by const reference!
        std::cerr << "[Exception Handler] Audit log: " << e.what() << "\n";
    }
}
```

Hành Vi Không Xác Định (Undefined Behavior): Bẫy Cắt Xén Đối Tượng Khi Bắt Ngoại Lệ Theo Giá Trị

Nếu khối bắt ngoại lệ được khai báo theo kiểu truyền giá trị (*Pass-by-value*):

```cpp
catch (std::exception e) // CRITICAL PITFALL: Object Slicing!
```

Giải phẫu cơ chế cắt xén dưới RAM:

1. Đối tượng ngoại lệ thực tế được ném ra là một thể hiện của lớp phái sinh `std::runtime_error` (mang thông điệp mô tả lỗi chi tiết trong heap buffer nội bộ).
2. Vì khối catch nhận theo giá trị, trình biên dịch buộc phải gọi Copy Constructor của lớp cơ sở: `std::exception::std::exception(const std::exception&)`.
3. Toàn bộ các trường dữ liệu và bản chất đa hình của lớp con bị **gọt bỏ hoàn toàn (Object Slicing)**.
4. Khi gọi `e.what()`, hàm ảo `what()` của `std::runtime_error` không được kích hoạt, mà chỉ gọi phiên bản mặc định vô nghĩa của `std::exception`.
5. **Quy tắc Vàng: LUÔN LUÔN BẮT NGOẠI LỆ BẰNG THAM CHIẾU HẰNG (`const ExceptionType& e`).**[a]

[a] Nguồn: OOP8-Exception.pdf - Trang 13-18: Bẫy Object Slicing khi bắt ngoại lệ

7.2 Giải Phóng Ngăn Xếp (Stack Unwinding) & Liên Minh RAII

Điều gì thực sự diễn ra trong không gian địa chỉ bộ nhớ khi lệnh `throw` được kích hoạt?

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Cơ Chế Stack Unwinding & Bảng Ngoại Lệ Zero-Cost

Trong các hệ thống C++ hiện đại (tuân thủ Itanium C++ ABI trên Linux hoặc Structured Exception Handling trên Windows), trình biên dịch áp dụng mô hình **Xử lý ngoại lệ Zero-Cost (Table-Driven Zero-Cost Exception Handling)**:

1. **Tại thời điểm bình thường (Zero Runtime Overhead):** Không có bất kỳ chi phí thực thi nào cho các khối `try`. Không cần lưu vết con trỏ hàm hay gọi lệnh kiểm tra. Mã máy chạy với tốc độ tối đa 100%.
2. **Khi lệnh `throw` ném ra ngoại lệ:**
   - Runtime C++ gọi hàm hỗ trợ nhân `_Unwind_RaiseException`.
   - Hệ thống thực hiện **Pha 1 (Search Phase):** Quét ngược Call Stack dựa trên bảng thông tin khung hàm **Call Frame Information (DWARF CFI trong phân vùng `.eh_frame`)** để kiểm tra xem có khối `catch` nào chấp nhận kiểu ngoại lệ này không mà *chưa thực hiện giải phóng tài nguyên.*
   - Nếu tìm thấy khối catch hợp lệ, hệ thống thực hiện **Pha 2 (Cleanup / Unwind Phase):** Quét ngược lại Call Stack lần thứ hai. Tại mỗi khung hàm (Stack Frame) bị thoát ra, **MỌI ĐỐI TƯỢNG ĐỊA PHƯƠNG TRÊN STACK ĐỀU ĐƯỢC TỰ ĐỘNG GỌI DESTRUCTOR** theo thứ tự ngược lại chính xác với thứ tự khởi tạo (LIFO)!
3. **Liên minh hoàn hảo với RAII:** Nếu toàn bộ tài nguyên (RAM, File Handlers, Socket, Mutex) được bao bọc trong các đối tượng RAII (`std::unique_ptr`, `std::ofstream`, `std::lock_guard`), thì cho dù ngoại lệ có bay qua 10 tầng khung hàm, toàn bộ bộ nhớ và file đều được đóng sạch sẽ 100%, **không bao giờ bị rò rỉ bộ nhớ**![a]

[a] Nguồn: OOP8-Exception.pdf - Trang 19-32: Stack Unwinding và cơ chế dọn dẹp bộ nhớ tự động

7.3 Từ Khóa `noexcept` & Tối Ưu Hóa STL Sống Còn Trong C++11

Chuẩn C++11 chính thức khai tử đặc tả ngoại lệ lỗi thời `throw(type)` và thay thế bằng từ khóa toàn năng **`noexcept`**.

7.3.1 Cam Kết Bất Biến `noexcept`

Khi một hàm được gắn nhãn `noexcept`, lập trình viên đưa ra một cam kết tuyệt đối với trình biên dịch và hệ thống rằng: *Hàm này bảo đảm không bao giờ để bất kỳ ngoại lệ nào thoát ra ngoài ranh giới của nó.*

```
void hardwareReset() noexcept {
    // Guaranteed never to leak any exception to the outer caller
}
```

Cảnh Báo Kỹ Thuật & Rủi Ro Biên Dịch: Hậu Quả Khi Vi Phạm Cam Kết `noexcept`

Nếu một ngoại lệ bất ngờ xuất hiện bên trong một hàm `noexcept` mà không được xử lý nội bộ:

- Hệ thống C++ **KHÔNG TIẾN HÀNH GIẢI PHÓNG STACK (Stack Unwinding)**.
- Runtime lập tức gọi ngay hàm hủy diệt tiến trình: **`std::terminate()`**.
- Ứng dụng lập tức bị sập (Aborted). Điều này giúp trình biên dịch không cần sinh thêm bảng giải phóng ngoại lệ cho hàm đó, tối ưu hóa đáng kể dung lượng mã máy nhị phân.

7.3.2 Bí Ẩn Tối Ưu Hiệu Năng Của `std::vector` Trong C++11

Tại sao việc thiếu từ khóa `noexcept` trên Move Constructor có thể làm giảm sút hiệu năng của cấu trúc dữ liệu STL nghiêm trọng?

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Toán Tử `noexcept(expr)` Và Chiến Lược Tái Cấp Phát Của `std::vector`

Xét kịch bản `std::vector<Widget>` bị đầy dung lượng (`size == capacity`) và phải cấp phát một vùng nhớ mới có dung lượng gấp đôi trên Heap:

1. Để đáp ứng **Cam Kết An Toàn Ngoại Lệ Mạnh Mẽ (Strong Exception Guarantee)**: Nếu có lỗi xảy ra trong quá trình di dời dữ liệu, trạng thái của vector cũ phải được bảo toàn nguyên vẹn 100% như trước khi gọi hàm.
2. Trình biên dịch sử dụng toán tử kiểm tra compile-time: `noexcept(Widget(std::declval<Widget&&>()))` (hoặc type trait `std::is_nothrow_move_constructible<Widget>`).
3. **Kịch bản 1 (Move Constructor CÓ `noexcept`):** Vector hoàn toàn yên tâm sử dụng Move Constructor để di dời từng phần tử từ vùng nhớ cũ sang vùng nhớ mới với chi phí hoán đổi con trỏ $O(1)$.
4. **Kịch bản 2 (Move Constructor THIẾU `noexcept`):** Vector lo sợ rằng nếu đang move dở chừng đến phần tử thứ 5 mà ngoại lệ xảy ra, các phần tử trước đó đã bị "đánh cắp" dữ liệu không thể khôi phục lại trạng thái ban đầu được! Do đó, **Vector BẮT BUỘC PHẢI DÙNG COPY CONSTRUCTOR!**
5. **Hậu quả:** Toàn bộ dữ liệu bị sao chép sâu chậm chạp qua nhiều thao tác cấp phát RAM phụ trợ, triệt tiêu hoàn toàn lợi thế tốc độ của C++11![a]

[a] Nguồn: OOP8-Exception.pdf - Trang 33-45: noexcept specifier và sự tương tác với STL Vector

7.4 Ngoại Lệ Trong Constructor & Destructor — Những Vùng Đất Cấm

7.4.1 Ngoại Lệ Trong Constructor: Hiểm Họa Rò Rỉ Con Trỏ Thô

Quy tắc pháp lập của C++: **Một đối tượng chỉ được coi là "đã tồn tại" sau khi Constructor của nó thực thi xong dòng lệnh cuối cùng!**

Nếu một ngoại lệ bị ném ra bên trong Constructor:

```
class VulnerableDevice {
private:
    int* rawMemoryBuffer;
    int portId;
public:
    VulnerableDevice(int port) : portId(port) {
        rawMemoryBuffer = new int[10000]; // Raw memory allocation
        if (port < 0) {
            throw std::invalid_argument("Invalid port configuration");
        }
    }

    ~VulnerableDevice() {
        delete[] rawMemoryBuffer; // NEVER CALLED IF CONSTRUCTOR THROWS!
    }
};
```

Hành Vi Không Xác Định (Undefined Behavior): Thảm Họa Rò Rỉ Vĩnh Viễn Trong Constructor

Vì ngoại lệ xảy ra khi Constructor chưa hoàn tất, đối tượng `VulnerableDevice` được coi là **chưa từng sinh ra**!

- Hệ thống **SẼ KHÔNG BAO GIỜ GỌI DESTRUCTOR `~VulnerableDevice()`**!
- Hậu quả: Dòng lệnh `delete[] rawMemoryBuffer` vĩnh viễn không được thực thi. Vùng nhớ 40 KB bị rò rỉ hoàn toàn trong RAM.
- **Giải Pháp Chuẩn Mực Tuyệt Đối:** Không bao giờ nắm giữ tài nguyên bằng con trỏ thô bên trong lớp. Luôn sử dụng Smart Pointer (`std::unique_ptr<int[]> rawMemoryBuffer`). Khi đó, cơ chế Stack Unwinding sẽ tự động kích hoạt Destructor của các thành viên con đã khởi tạo thành công trước đó![a]

[a] Nguồn: OOP8-Exception.pdf - Trang 46-52: Ngoại lệ trong Constructor và kỹ thuật phòng vệ

7.4.2 Kỹ Thuật Function Try Block Trong Constructor

Làm thế nào để bắt được ngoại lệ xảy ra ngay bên trong Member Initializer List của Constructor? C++ cung cấp cú pháp **Function Try Block**:

```
class ServerNode {
private:
    DatabaseConnection dbConn;
public:
    // Function Try Block covers the member initializer list!
```

```
    ServerNode(const std::string& connString)
    try : dbConn(connString) {
        // Constructor body
    }
    catch (const std::exception& e) {
        std::cerr << "[Initialization Error] Database connection failed: " << e.
    what() << "\n";
        // ISO C++ RULE: At the end of catch block in Constructor,
        // the exception is AUTOMATICALLY RE-THROWN! You cannot suppress it!
    }
};
```

7.4.3 Luật Cấm Tuyệt Đối: Ngoại Lệ Thoát Ra Khỏi Destructor

Trong C++11, mọi Destructor mặc định tự động mang thuộc tính ngầm định là **`noexcept(true)`**.

Hành Vi Không Xác Định (Undefined Behavior): Lý Do Hệ Thống Bị Tiêu Diệt Tức Thì (Two Active Exceptions Collision)

Giả sử một ngoại lệ đang bay lên và Call Stack đang tiến hành dọn dẹp (*Stack Unwinding*). Trong quá trình đó, Destructor của một đối tượng địa phương trên Stack được kích hoạt.
Nếu Destructor đó lại bất cẩn để một ngoại lệ thứ hai thoát ra ngoài:

- Hệ điều hành C++ rơi vào tình huống nghịch lý không thể giải quyết: **Tồn tại đồng thời HAI ngoại lệ đang hoạt động song song**.
- C++ không cho phép hai ngoại lệ cùng bay trên Call Stack. Runtime lập tức gọi ngay **`std::terminate()`** để hủy diệt tiến trình ngay tại chỗ![a]

Quy tắc Bất Biến: Destructor phải bắt và nuốt trọn mọi ngoại lệ nội bộ bên trong một khối `try-catch`, tuyệt đối không để bất kỳ lỗi nào lọt ra ngoài!

[a]Nguồn: OOP8-Exception.pdf - Trang 53-58: Luật cấm ngoại lệ trong Destructor

7.5 Ba Cấp Độ Cam Kết An Toàn Ngoại Lệ (Exception Safety Guarantees)

Một hệ thống phần mềm C++ chuẩn mực phải phân định rõ ba cấp độ an toàn ngoại lệ theo phân loại của David Abrahams:

1. **Cam Kết Cơ Bản (Basic Guarantee):** Nếu một ngoại lệ xảy ra, hệ thống bảo đảm **không có tài nguyên nào bị rò rỉ** (không rò rỉ RAM, handle file, hay socket) và tất cả các đối tượng đều ở trạng thái hợp lệ nội tại (*Valid Invariants*), dù dữ liệu có thể bị biến đổi so với ban đầu.
2. **Cam Kết Mạnh Mẽ (Strong Guarantee / Commit-or-Rollback):** Thao tác mang tính nguyên tử (*Atomic*): Hoặc là thành công trọn vẹn, hoặc nếu có ngoại lệ xảy ra thì trạng thái của toàn bộ hệ thống được phục hồi nguyên vẹn

chính xác 100% như trước khi gọi hàm!

- *Phương pháp hiện thực hóa kinh điển:* Thao tác trên một bản sao tạm thời trước; chỉ cập nhật vào trạng thái thật bằng các phép toán `noexcept` sau khi mọi rủi ro đã vượt qua an toàn (Copy-and-Swap).

3. **Cam Kết Không Bao Giờ Thất Bại (Nothrow / No-fail Guarantee):** Hàm cam đoan 100% thành công và không bao giờ ném ra bất kỳ ngoại lệ nào. Đây là cam kết bắt buộc đối với:
   - Mọi Destructor.
   - Các hàm giải phóng tài nguyên hệ thống (`free`, `delete`, `close`).
   - Các hàm hoán vị dữ liệu (`std::swap`) và Move Constructors.[1]

7.5.1 Nghiên Cứu Điển Hình Kiến Trúc: Tại Sao `std::stack::pop()` Trả Về `void`?

Tại sao trong thư viện STL, hàm `std::stack::pop()` chỉ đơn thuần xóa phần tử đỉnh mà không trả về giá trị của phần tử đó (buộc lập trình viên phải gọi `top()` trước rồi mới gọi `pop()`)?

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Lý Do Đằng Sau Thiết Kế `std::stack::pop()`

Giả sử ta thiết kế hàm `T pop()` trả về đối tượng theo giá trị:

```
template <typename T>
T Stack<T>::pop() {
    T result = elements.top();
    elements.pop_internal(); // Element removed from stack
    return result; // RETURN BY VALUE TRIGGERS COPY CONSTRUCTOR!
}
```

Kịch bản thảm họa:

1. Phần tử đỉnh đã bị xóa khỏi Stack.
2. Khi trả về cho hàm gọi, phép sao chép `return result;` kích hoạt Copy Constructor của kiểu `T`.
3. Nếu kiểu `T` bị cạn kiệt bộ nhớ và ném ra ngoại lệ `std::bad_alloc`: Phần tử đã bị xóa mất khỏi Stack, nhưng hàm gọi chưa hề nhận được dữ liệu!
4. Dữ liệu bị **mất trắng vĩnh viễn**! Không thể khôi phục lại trạng thái cũ → **Cam Kết An Toàn Mạnh Mẽ bị phá vỡ hoàn toàn!**
5. **Kết luận:** Bằng cách tách biệt `top()` (chỉ đọc tham chiếu) và `pop()` (xóa `noexcept`), STL bảo đảm trọn vẹn Cam Kết An Toàn Mạnh Mẽ!

[1] Nguồn: OOP8-Exception.pdf - Trang 59-68: Ba cấp độ an toàn ngoại lệ của David Abrahams

7.6 Kiến Trúc Tách Rời Trong Thư Viện Chuẩn C++ (STL)

Thư viện Chuẩn C++ (**Standard Template Library — STL**) do Alexander Stepanov sáng tạo là kiệt tác vĩ đại nhất của sự kết hợp giữa Lập trình Tổng quát và Tư duy Hướng Đối tượng.

7.6.1 Kiến Trúc Tách Rời (Decoupled Architecture)

Trong các thư viện hướng đối tượng cổ điển, thuật toán thường được đóng gói trực tiếp vào bên trong các lớp thùng chứa (`Container::sort()`). Stepanov nhận ra rằng thiết kế này dẫn đến sự bùng nổ tổ hợp: Với N cấu trúc dữ liệu và M thuật toán, ta sẽ phải hiện thực hóa $N \times M$ phương thức riêng biệt!



Figure 7.1: Kiến trúc ba tầng tách rời (Decoupled Architecture) của Alexander Stepanov trong STL.

Bằng cách phân tách hoàn toàn Thuật toán và Cấu trúc dữ liệu thông qua giao diện trung gian là **Bộ lặp (Iterators)**, N cấu trúc dữ liệu và M thuật toán chỉ đòi hỏi đúng $N + M$ thành phần độc lập!

7.6.2 Năm Hạng Bộ Lặp & Kỹ Thuật Tag Dispatching

STL phân loại Bộ lặp thành 5 hạng dựa trên năng lực truy cập bộ nhớ phần cứng:

1. `std::input_iterator_tag`: Đọc tuần tự một chiều một lần (ví dụ: luồng dữ liệu `std::cin`).
2. `std::output_iterator_tag`: Ghi tuần tự một chiều một lần (ví dụ: luồng dữ liệu `std::cout`).
3. `std::forward_iterator_tag`: Đọc/ghi tuần tự nhiều lần (`std::forward_list`).
4. `std::bidirectional_iterator_tag`: Di chuyển tiến và lùi tuần tự (`std::list`, `std::map`, `std::set`).
5. `std::random_access_iterator_tag`: Nhảy tới bất kỳ vị trí nào với độ phức tạp $O(1)$ (`std::vector`, `std::deque`, mảng thô).

Kỹ Thuật Tag Dispatching Tại Compile-Time: Khi gọi hàm dịch chuyển bộ lặp `std::advance(it, n)`, STL sử dụng cơ chế nạp chồng dựa trên tag để chọn thuật toán tối ưu nhất mà **hoàn toàn không tốn một chu kỳ CPU nào tại runtime**:

```
// Optimal implementation for Random Access Iterator: Single CPU instruction in O
    (1)
template <typename RandomAccessIterator, typename Distance>
```

```
void advance_impl(RandomAccessIterator& it, Distance n, std::
    random_access_iterator_tag) {
    it += n;
}

// Fallback implementation for Bidirectional Iterator: Iterative step in O(N)
template <typename BidirectionalIterator, typename Distance>
void advance_impl(BidirectionalIterator& it, Distance n, std::
    bidirectional_iterator_tag) {
    while (n > 0) { ++it; --n; }
    while (n < 0) { --it; ++n; }
}

template <typename Iterator, typename Distance>
void advance(Iterator& it, Distance n) {
    // Compile-time tag lookup selects the exact overload with ZERO runtime branch
     overhead:
    advance_impl(it, n, typename std::iterator_traits<Iterator>::iterator_category
    ());
}
```

7.6.3 Kiến Trúc Quản Trị Bộ Nhớ: Bộ Cấp Phát Tùy Biến (Allocators & Rebind)

Một thùng chứa STL như `std::list<T>` không cấp phát trực tiếp các đối tượng `T` rời rạc. Trong thực tế, nó cần cấp phát các nút danh sách:

```
template <typename T>
struct ListNode {
    ListNode* next;
    ListNode* prev;
    T data;
};
```

Nếu người dùng truyền vào một bộ cấp phát kiểu `std::allocator<T>`, làm thế nào để thùng chứa có thể cấp phát bộ nhớ cho `ListNode<T>`?

Câu trả lời nằm ở **Cơ Chế Tái Liên Kết (The Rebind Mechanism)**:

```
template <typename T, typename Allocator = std::allocator<T>>
class CustomList {
    // Rebind Allocator from type T to ListNode<T>:
    using NodeAlloc = typename std::allocator_traits<Allocator>::template
    rebind_alloc<ListNode<T>>;
    NodeAlloc nodeAllocator;
};
```

Đây là minh chứng đỉnh cao cho sự trừu tượng hóa toàn vẹn của C++: Quản trị bộ nhớ vật lý được phân tách hoàn toàn khỏi cấu trúc dữ liệu và logic thuật toán.[2]

[2]Nguồn: Thingking in C++ vol 2 - Bruce Eckel.pdf - Part 2: Standard C++ Library & STL Architecture

Lời Kết: Con Đường Làm Chủ C++ Hiện Đại

Trải qua 7 chương sách kinh điển của **"The OOP C++ Bible"**, chúng ta đã cùng nhau du hành từ tầng kiến trúc bộ nhớ vật lý thấp nhất (Stack, Heap, Cache, Calling Conventions, ABI Assembly Dispatching, VTable) lên đến những tầng trừu tượng thanh lịch nhất của khoa học máy tính hiện đại (RAII, Move Semantics, Variadic Templates, CRTP, SFINAE, và Exception Safety Guarantees).

C++ không phải là một ngôn ngữ dễ dãi. Nó không che giấu sự phức tạp của máy tính sau một máy ảo chậm chạp. Nó trao toàn bộ quyền kiểm soát tối thượng vào tay lập trình viên, và như Bjarne Stroustrup từng đúc kết:

"C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do, it blows your whole leg off."

Nhưng khi bạn đã nắm vững các nguyên lý "dưới gầm máy" được trình bày trong cuốn sách này, bạn không còn sợ hãi những bẫy ngầm của bộ nhớ nữa. Bạn đã nắm trong tay sức mạnh để kiến tạo nên những hệ thống phần mềm có hiệu năng vô song, an toàn tuyệt đối và trường tồn cùng thời gian.

Chúc các bạn luôn giữ vững niềm đam mê và tự hào trên con đường trở thành một **C++ Master**!